

FRIttata: A FRI-based Polynomial Commitment Scheme for Distributed Proof Generation

Hua Xu¹ , Mariana Gama¹ , Emad Heydari Beni^{1,2}  and Jiayi Kang¹ 

¹ COSIC, KU Leuven, Leuven, Belgium

² Nokia Bell Labs, Antwerp, Belgium

Abstract. We present the first horizontally scalable polynomial commitment scheme (PCS) that is both transparent and plausibly post-quantum (PQ) secure. This PCS can be combined with the distributed polynomial interactive oracle proof (PIOP) introduced in Pianist (IEEE S&P 2024), which achieves linear scalability by encoding witnesses using bivariate polynomials. While Pianist and other scalable SNARK systems offer strong performance profiles, they rely on trusted setup ceremonies and cryptographic assumptions that are not PQ secure, e.g., pairing-based primitives. In contrast, we present a bivariate PCS based on FRI, which, when used to compile the Pianist PIOP, achieves a transparent and plausibly PQ alternative. Distributed FRI has a high communication cost. Therefore, we introduce *Fold-and-Batch*, a customizable technique that applies partial folding locally before performing batched FRI centrally. We formally prove the security of our constructions and provide an implementation for three variants of distributed FRI, accompanied by thorough performance evaluations. Our results show that Fold-and-Batch reduces communication overhead compared to existing distributed FRI approaches, while preserving scalability and maintaining moderate proof sizes. To our knowledge, this is the first horizontally scalable PCS that simultaneously achieves transparency, plausible PQ security, and a tunable tradeoff between efficiency, verifier cost, and communication.

Keywords: Zero-knowledge proofs · Horizontal scalability · FRI

1 Introduction

A *Succinct Non-Interactive Argument of Knowledge*, also called a SNARK, is a short proof that a statement is true, and any prover who can generate a valid SNARK must know a corresponding “witness”. SNARKs are *succinct*, meaning that the proof length and the time needed to verify it are poly-logarithmic in the size of the statement. Additionally, the proving process is *non-interactive*, i.e., the prover can generate the proof without communicating with the verifier. If the proof does not reveal any information about the witness beyond the statement itself, we call it a *zero-knowledge SNARK* (zk-SNARK). While zk-SNARKs have enabled many real-world applications, proof generation in existing schemes remains computationally expensive, and proving large statements on a single machine requires a significant amount of memory. *Plonk* [GWC19], one of the most popular zk-SNARK proof systems, requires 121 seconds and 200 GB of memory to prove a statement expressed using 2^{25} gates [LXZ⁺24]. This severely limits the applications of SNARKs.

One solution to this problem is to generate SNARK proofs distributively by dividing the data and computational tasks among a cluster of machines. However, most SNARK proof

E-mail: hua.xu@esat.kuleuven.be (Hua Xu), mariana.botelhodagama@esat.kuleuven.be (Mariana Gama), emad.heydaribeni@esat.kuleuven.be (Emad Heydari Beni), jiayi.kang@esat.kuleuven.be (Jiayi Kang)



systems were not designed to be distribution-friendly, and oftentimes the SNARK protocol must be adapted for efficient distributed computation. DIZK [WZC⁺18], based on the Groth16 [Gro16] proof system, was the first effort to build a distributed zk-SNARK. Using DIZK, one can prove statements that are 100 times larger than when using a monolithic prover; however, it requires a linear amount of communication between the prover nodes due to the interleaving pattern of the FFT algorithm.

Pianist [LXZ⁺24] is another distributed proof system based on *Plonk*. Instead of using univariate polynomials to encode the prover’s witness, *Pianist* employs a novel use of bivariate polynomials to encode the witnesses. This allows the cluster of provers to compute FFTs only on their local polynomials, thereby avoiding the linear communication cost associated with distributing the FFT. *Pianist* is perfectly scalable in both proving time and memory usage; that is, if we double the number of prover nodes, the proving time and memory usage of each node are halved.

Plonk and *Pianist* both use the KZG polynomial commitment scheme (PCS) [KZG10], whose security relies on the discrete logarithm problem. KZG requires a trusted setup procedure in which a secret value is used to generate parameters for the proof computation. This secret value must be destroyed at the end of the setup stage; otherwise, it can be used to render the proof system completely insecure. It is therefore desirable to have *transparent* proof systems that do not require such trusted setups. Among these are SNARKs based on FRI [BBHR18], the *Fast Reed-Solomon Interactive Oracle Proofs of Proximity*. In addition to being transparent, FRI is also plausibly post-quantum secure and is used in most of the post-quantum zk-SNARK systems deployed today.

This paper explores the possibility of adapting *Pianist*’s distribution technique to the setting of FRI-based SNARKs. We do so by constructing a bivariate polynomial commitment scheme that is simultaneously *horizontally scalable*, *transparent*, and *plausibly post-quantum secure*, and then use it to compile *Pianist*’s distributed polynomial interactive oracle proof (PIOP) into a public-coin interactive argument of knowledge for general circuits.

1.1 Related Work

Besides DIZK and *Pianist*, numerous other constructions for distributed SNARKs have been proposed. We review them below and summarize their properties in Table 1 (Section 1.2). In Table 5 (Appendix A), we present further details regarding the communication patterns and security assumptions of these related works.

HyperPianist [LZL⁺25a] is a distributed SNARK based on *Pianist* and HyperPlonk [CBBZ23]. By encoding the witness using a multilinear polynomial evaluated on a Boolean hypercube, similarly to Hyperplonk, HyperPianist is able to avoid FFT computations and has a horizontally scalable prover. The communication cost for each machine is logarithmic in the overall circuit size. Moreover, instantiating the HyperPianist PIOP with a distributed variant of the Dory PCS [Lee21] results in a transparent (but not post-quantum secure) SNARK system. Cirrus [WSVZ24] is another distributed SNARK based on HyperPlonk [CBBZ23]. It features *accountability*, which means the master node can identify malicious worker nodes. Cirrus has a horizontally scalable prover and a per-machine communication cost that is logarithmic in the size of the sub-circuits.

Apart from Plonk-based SNARKs, there is also a line of work distributing SNARKs using the Rank-1 constraint system (R1CS). DIZK [WZC⁺18] is the first distributed SNARK for R1CS. Hekaton [RMH⁺24] is a distributed SNARK based on Mirage [KPPS20], which is a variant of Groth16 [Gro16]. Hekaton uses a *divide-and-aggregate* framework, where each prover node generates a separate SNARK proof for its sub-circuit. Then, these proofs are aggregated into a single, succinct proof. Hekaton has a horizontally scalable prover, a constant amount of communication, and a logarithmic proof size. Soloist [LZL⁺25b] is the latest distributed SNARK for R1CS. It has a horizontally scalable prover that runs in

time quasi-linear in the size of the sub-circuit, with a constant amount of communication per machine and a constant proof size.

Note that only one of the mentioned schemes is transparent, and none is plausibly post-quantum, as their security relies on the discrete logarithm problem (similarly to DIZK and *Pianist*); to the best of our knowledge, there is no horizontally scalable SNARK for *general circuits* that satisfies both properties. There is, however, such a proof system known as DeVirgo [XZC⁺22] with the restriction that it can only prove statements represented by *data-parallel circuits*, meaning that all the sub-circuits handled by prover nodes are identical, with no connecting wires among them. DeVirgo is a distributed SNARK based on the transparent and plausibly post-quantum SNARK Virgo [ZXZS20] and using a FRI-based PCS, whose security relies on the collision resistance of hash functions. It has a horizontally scalable prover, but the communication cost per machine is relatively high due to the need for prover nodes to exchange witness data. As a result of using FRI, deVirgo also produces relatively large proofs, which can be significantly reduced using recursive verification with Groth16.

1.2 Our Setting and Contribution

Starting point: the *Pianist* proving system. *Pianist* consists of two main building blocks that are combined to obtain a distributed SNARK with perfect scalability:

1. A distributed version of the Plonk polynomial IOP (Interactive Oracle Proof).
2. A distributed version of the KZG polynomial commitment scheme ([LXZ⁺24], Protocol 2 DKZG), which allows a prover to
 - (a) Commit to a bivariate polynomial of the form

$$F(X, Y) = \sum_{i=0}^{M-1} F_i(X) R_i(Y)$$

where $\deg F_i < T$ for some degree bound T and R_i 's are Lagrange polynomials defined on the M -th roots of unity.

- (b) Open $F(X, Y)$ at a point $(\alpha, \beta) \in \mathbb{F}^2$ while providing an evaluation proof.

In *Pianist* (as well as in this work), a master node is responsible for coordinating computations and communication among the worker nodes. Additionally, the master node also functions as one of the worker nodes. The master and worker nodes form a star topology, with the master node sitting in the center, so there is no communication between two non-master nodes. The entire circuit is evenly distributed among the worker nodes, meaning that every node will compute a proof for a subcircuit of the same size. Note that this communication pattern is shared by most of the distributed schemes mentioned in Section 1.1 except for Cirrus and deVirgo, where there is communication between worker nodes, and DIZK, where there is no master node and all nodes communicate with each other. We refer to Table 5 (Appendix A) for illustrations of these communication patterns.

Dealing with adversarial nodes. *Pianist* formalizes the notion of *robust collaborative proving scheme* (RCPS) and shows that, when dealing with data-parallel circuits, the master node is able to identify malicious worker nodes attempting to sabotage the entire proof [LXZ⁺24, Theorem 5]. We expect that this property can also be achieved in our setting by having the master node verify the partial FRI proofs from all worker nodes, but leave a formal analysis (and an extension to general circuits) for future work. Therefore, in our construction, the presence of any malicious node will result in a proof that does not verify due to the soundness of the overall protocol.

Our Contribution. The contributions of this work are as follows:

- We present a bivariate polynomial commitment scheme based on FRI (denoted as PCS), built upon a distributed FRI subprotocol (denoted as DFRI). As an extension of the univariate FRI-based PCS [VP19], our PCS scheme retains key features such as transparency and plausible post-quantum security, relying on the existence of collision-resistant hash functions. We formally establish the security of PCS, including its completeness, (polynomial and evaluation) binding properties, and knowledge soundness that is inherited from knowledge soundness of DFRI.
- We construct a distributively computable public-coin interactive argument of knowledge for general circuits that is transparent and plausibly post-quantum secure. This is built by compiling *Pianist*'s bivariate IOP with our PCS scheme. We also provide a security analysis of the resulting protocol. A summary of the asymptotic performance of our results in comparison with other distributed schemes is presented in Table 1.
- We describe three instantiations for DFRI: Parallel FRI, Distributed Batched FRI [Che24], and Fold-and-Batch. While Parallel FRI yields a large proof size and Distributed Batched FRI incurs a large communication cost that is linear in the overall circuit size, our newly proposed method, Fold-and-Batch, takes advantage of both to reduce communication costs while maintaining a relatively small proof size. Furthermore, we prove all three DFRI instantiations have knowledge soundness as interactive arguments.
- We implement the three DFRI instantiations in Rust based on winterfell's FRI implementation¹. We present the measurements of the prover, verifier, and communication costs, and show that Fold-and-Batch has an efficient prover-verifier tradeoff.

Table 1: A comparison of distributed proof systems. N : size of the overall circuit. M : number of worker machines. $T = \frac{N}{M}$: size of each sub-circuit held by a worker node. $K \in [1, T]$: Fold-and-Batch folding parameter. “Gen.”: general circuits. “Trans.”: transparent setup. “PQ”: plausibly post-quantum. “Overall runtime”: overall prover runtime. “Total comm.”: total amount of communication between the master and worker nodes. “Our PCS” refers to our bivariate PCS instantiated with Fold-and-Batch.

System	Gen.	Trans.	PQ	$\mathcal{P}_i$ time	Overall runtime	Total comm.	$\mathcal{V}$ time
DIZK	✓	✗	✗	$O(T \log^2 T)$	$O(T \log^2 T)$	$O(N)$	$O(1)$
deVirgo	✗	✓	✓	$O(T \log T)$	$O(T \log T)$	$O(N)$	$O(\log^2 N)$
Cirrus	✓	✗	✗	$O(T)$	$O(T)$	$O(M \log T)$	$O(\log N)$
Pianist	✓	✗	✗	$O(T \log T)$	$O(T \log T)$	$O(M)$	$O(1)$
Hekaton	✓	✗	✗	$O(T \log T)$	$O(T \log T)$	$O(M)$	$O(\log M)$
HyperPianist ^K	✓	✗	✗	$O(T)$	$O(T + M \log T)$	$O(M \log N)$	$O(\log N)$
HyperPianist ^D	✓	✓	✗	$O(T)$	$O(T + M \log T)$	$O(M \log N)$	$O(\log N)$
Soloist	✓	✗	✗	$O(T \log T)$	$O(T \log T)$	$O(M)$	$O(1)$
Our PCS + Pianist PIOP	✓	✓	✓	$O(T \log T)$	$O(MK + M \log \frac{T}{K} \log T + T \log T)$	$O(MK + M \log \frac{T}{K} \log T)$	$O(M \log \frac{T}{K} \log T + \log^2 K)$

¹<https://github.com/facebook/winterfell>

2 Preliminaries

2.1 Notations

For any positive integer k , $[k]$ denotes $\{1, \dots, k\}$. Let $\mathbb{F}$ be a finite field. Throughout the paper, we assume $\mathbb{F}$ is at least 256-bit. This is necessary to guarantee that our protocol has at least 128 bits of security. Let N be the size of the arithmetic circuit over $\mathbb{F}$ for which we would like to compute a proof. Let M be the number of machines participating in the protocol. We use P_0 for the master node, and $\{P_i\}_{1 \leq i \leq M-1}$ for worker nodes. Each node computes a proof for a subcircuit of size $T := \frac{N}{M}$.

Let $\text{RS}[\mathbb{F}, D, T]$ denote the following set of Reed-Solomon codewords:

$$\text{RS}[\mathbb{F}, D, T] := \{(F(x))_{x \in D} \mid F(X) \in \mathbb{F}[X]_{<T}\},$$

where D is the *evaluation domain* of size $|D| = n > T$. The *rate* of the Reed-Solomon code is $\rho := \frac{T}{n}$. This paper considers $D = \mu_n$, the n -th roots of unity with $n = 2^k$ being a power of 2. We also use the shorthand RS to denote $\text{RS}[\mathbb{F}, D, T]$. In FRI, the inverse of the code rate, ρ^{-1} , is referred to as the *blow-up factor*. In practice, the blow-up factor is always a power of 2, commonly set to 4 or 8 [0xP21]. For functions $f, g : D \rightarrow \mathbb{F}$, we use $\Delta(f, g)$ to denote the *relative Hamming distance* between f and g , i.e.,

$$\Delta(f, g) := \frac{1}{n} \cdot |\{x \in D \mid f(x) \neq g(x)\}|.$$

Let $\delta \in [0, 1)$. A function $f : D \rightarrow \mathbb{F}$ is δ -close to RS if there is a polynomial $F \in \mathbb{F}[X]_{<T}$ such that $\Delta(f, F) < \delta$. In this work, we assume $\delta < \frac{1-\rho}{2}$, the unique decoding radius of RS . This means, if f is δ -close to RS , then there is a unique polynomial $F \in \mathbb{F}[X]_{<T}$ in the ball of radius δ around f . For any function $f : D \rightarrow \mathbb{F}$, define $\text{RS.Decode}(f)$ to be this unique polynomial $F \in \mathbb{F}[X]_{<T}$ if f is δ -close to RS . We also denote the evaluation $\text{RS.Decode}(f)|_D$ as $\text{RS.Decode}(f)$. Define $\text{RS.Decode}(f)$ to be f if it is δ -far.

Degrees of Scalability. Throughout the paper, we use the term *horizontal scalability* in the following (informal) manner: we say that a distributed proving scheme is *horizontally scalable*, if the proving time and memory usage of the prover can be effectively reduced by increasing the number of participating machines. We say a distributed proving scheme is *perfectly scalable* or *linearly scalable*, if the proving time and memory usage of the prover are reduced by a factor of d when the number of participating machines increases by a factor of d . Roughly speaking, horizontal scalability refers to practical scalability, while linear scalability represents optimal scalability.

2.2 Interactive Proofs and Arguments

Definition 1 (Relations). A *relation* $\mathcal{R}$ is a subset of $\{0, 1\}^* \times \{0, 1\}^*$. If $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$, then $\mathbf{x}$ is an *instance* of $\mathcal{R}$ and $\mathbf{w}$ is a *witness* for the instance $\mathbf{x}$.

Definition 2 (Languages). The *language* $\mathcal{L}_{\mathcal{R}}$ associated to a relation $\mathcal{R}$ is the set of strings $\mathbf{x} \in \{0, 1\}^*$ such that $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$ for some $\mathbf{w} \in \{0, 1\}^*$.

Definition 3 (Interactive Arguments of Knowledge). Let $\mathcal{R}$ be a relation. An *interactive argument of knowledge* for $\mathcal{R}$ is a pair of probabilistic polynomial time interactive algorithms $(\mathcal{P}, \mathcal{V})$ satisfying the following conditions:

- **Completeness** Let $\langle \mathcal{P}(\mathbf{x}, \mathbf{w}), \mathcal{V}(\mathbf{x}) \rangle$ denote the output by $\mathcal{V}$ after the interaction between $\mathcal{P}$ and $\mathcal{V}$ where $\mathcal{P}$ has input $\mathbf{x}, \mathbf{w}$ and $\mathcal{V}$ has input $\mathbf{x}$. If $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$, then $\langle \mathcal{P}(\mathbf{x}, \mathbf{w}), \mathcal{V}(\mathbf{x}) \rangle = \text{accept}$ with probability 1.

- **Knowledge Soundness** There is a polynomial time algorithm E with the following property: Let $\mathbf{x} \in \{0, 1\}^*$ and let $\mathcal{P}^*$ be any polynomial-time interactive algorithm. If given input $\mathbf{x}$ and access to messages from $\mathcal{P}^*$ during the protocol, E outputs a $\mathbf{w} \in \{0, 1\}^*$ satisfying $\Pr[\langle \mathcal{P}^*(\mathbf{x}), \mathcal{V}(\mathbf{x}) \rangle = \text{accept} \wedge (\mathbf{x}, \mathbf{w}) \notin \mathcal{R}] = \text{negl}(\lambda)$.

Definition 4 (Public-coin). An interactive protocol is *public-coin* if all the messages sent by the verifier are uniformly random and independent of the prover’s messages. Intuitively, all verifier’s messages are uniformly random coin tosses that could have been sampled by a trusted third party.

Definition 5 (Interactive Oracle Proofs). An *interactive oracle proof* for a relation $\mathcal{R}$ with δ -completeness and ε -soundness is a pair of interactive algorithms $\Pi = (\mathcal{P}, \mathcal{V})$ such that:

- For $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$, $\mathcal{P}$ receives $(\mathbf{x}, \mathbf{w})$ as input and $\mathcal{V}$ receives $\mathbf{x}$ as input before the start of the protocol.
- During each round of interaction, $\mathcal{P}$ sends $\mathcal{V}$ an oracle message m which $\mathcal{V}$ can query at any position. Then, $\mathcal{V}$ responds with a message c . At the end of the protocol, $\mathcal{V}$ outputs either **accept** or **reject**.
- (δ -completeness) For all $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$, we have $\Pr[\langle \mathcal{P}(\mathbf{x}, \mathbf{w}), \mathcal{V}(\mathbf{x}) \rangle = \text{accept}] \geq \delta$, where the probability is taken over the random coins of $\mathcal{V}$.
- (ε -soundness) If $\mathbf{x} \notin \mathcal{L}_{\mathcal{R}}$, then for any unbounded interactive algorithm $\mathcal{P}^*$ we have

$$\Pr[\langle \mathcal{P}^*(\mathbf{x}), \mathcal{V}(\mathbf{x}) \rangle = \text{accept}] \leq \varepsilon,$$

where the probability is taken over the random coins of $\mathcal{V}$.

Definition 6 ([BGK⁺23], Definition 3.13). A k -round (public-coin) IOP for a relation $\mathcal{R}$ has *round-by-round knowledge soundness* with knowledge error ε_k if there exists a polynomial-time extractor Ext and a (not necessarily efficiently computable) “doomed set” $\mathcal{D}$ containing pairs $(\mathbf{x}, \tau)$ where $\mathbf{x}$ is an input and τ is a partial or complete transcript such that:

1. $(\mathbf{x}, \emptyset) \in \mathcal{D}$ for all possible input $\mathbf{x}$, regardless of whether $\mathbf{x} \in \mathcal{L}_{\mathcal{R}}$ or not.
2. For any possible input $\mathbf{x}$ and any complete transcript τ , if $(\mathbf{x}, \tau) \in \mathcal{D}$ then $\mathcal{V}(\mathbf{x}, \tau) = \text{reject}$.
3. If $i \in [k]$ and τ is a $(i - 1)$ -round partial transcript such that $(\mathbf{x}, \tau) \in \mathcal{D}$, then for every potential prover next message m , if $\Pr_{c \leftarrow C_i}[(\mathbf{x}, \tau_{i-1} \parallel m \parallel c) \notin \mathcal{D}] > \varepsilon_k$, then $\text{Ext}(\mathbf{x}, \tau, m)$ outputs a valid witness for $\mathbf{x}$.

2.3 The FRI low-degree test and Batched FRI

The FRI low-degree test was introduced in [BBHR18] to prove that a Merkle-committed function $f : D \rightarrow \mathbb{F}$ is δ -close to a polynomial of degree $< T$, i.e. δ -close to $\text{RS}[\mathbb{F}, D, T]$. The FRI protocol consists of a *folding phase* and a *query phase*. In the folding phase, $\mathcal{P}$ folds f into a smaller instance so that it takes less work for $\mathcal{V}$ to check that f is close to a polynomial of low degree. In the query phase, $\mathcal{V}$ sends several challenges to $\mathcal{P}$, which $\mathcal{P}$ must answer correctly for $\mathcal{V}$ to accept. These queries check that $\mathcal{P}$ has folded f honestly. We include the full FRI protocol in Appendix B for completeness.

Batched FRI. The batched FRI protocol [BCI⁺20] proves proximity to **RS** for multiple polynomials $f_1, \dots, f_k \in \mathbb{F}[X]$ by running a single instance of FRI. The batched FRI procedure consists of the following steps:

1. $\mathcal{P}$ sends the Merkle commitments $\{\text{MT.Commit}(f_i)\}_{i=1}^k$ to $\mathcal{V}$.
2. $\mathcal{V}$ samples a random challenge $\theta \xleftarrow{\$} \mathbb{F}$ and sends it to $\mathcal{P}$.
3. $\mathcal{P}$ computes the random linear combination $f(X) := \sum_{i=1}^k \theta^i f_i(X)$. $\mathcal{P}$ and $\mathcal{V}$ run FRI on $f(X)$. During the query phase, $\mathcal{V}$ performs some additional checks for each FRI query g :
 - (a) $\mathcal{V}$ queries $f_i(X)$ at g for all i .
 - (b) $\mathcal{V}$ checks that the opening proof (Merkle authentication path) for each $f_i(g)$ is valid and that $f(g) = \sum_{i=1}^k \theta^i f_i(g)$.
4. If $\mathcal{V}$ accepts the FRI proof for f and the additional checks pass, $\mathcal{V}$ outputs **accept**. Otherwise, $\mathcal{V}$ outputs **reject**.

The soundness of batched FRI is proven in [BCI⁺20], Theorem 8.3. Informally, it says that if any of the functions f_i is δ -far from **RS**, then with high probability, the batched FRI verifier will reject.

2.4 A FRI-based Univariate PCS

Vlasov and Panarin [VP19] proposed a univariate polynomial commitment scheme based on FRI, avoiding the trusted setup required in KZG-based schemes. It consists of the following algorithms:

- **Setup**(1^λ) $\rightarrow$ **pp**: Generates the public parameters **pp** = $(\mathbb{F}, T, l, \rho, \delta)$ containing the field $\mathbb{F}$, degree bound T , number of queries l , code rate ρ and the proximity parameter δ for FRI.
- **Commit**($f, \mathbf{pp}$) $\rightarrow \mathcal{C}$: Given any function $f : D \rightarrow \mathbb{F}$, outputs the Merkle commitment $\mathcal{C} = \text{MT.Commit}(f)$.
- **Open**($\mathcal{C}, \alpha, \mathbf{pp}$) $\rightarrow$ **accept** or **reject**:
 1. It is assumed that $\alpha \notin D$. $\mathcal{P}$ computes $z := f(\alpha)$ and sends it to $\mathcal{V}$.
 2. $\mathcal{P}$ and $\mathcal{V}$ run the FRI low-degree test on the polynomial $q(X) := \frac{f(X)-z}{X-\alpha}$. The oracle queries for $q(X)$ are provided through the oracle of $f(X)$. In other words, if $\mathcal{V}$ would like to query $q(x)$ for some $x \in D$, then $\mathcal{V}$ queries $f(x)$ and computes $\frac{f(x)-z}{x-\alpha}$. If $\mathcal{V}$ accepts the FRI proof, then it outputs **accept**. Otherwise, $\mathcal{V}$ outputs **reject**.

The FRI low-degree test in **Open** proves that $q(X) = \frac{f(X)-z}{X-\alpha}$ is δ -close to **RS**. In fact, this implies that $f(X)$ must also be δ -close to **RS**. In the end, the verifier is convinced that $f(X)$ is δ -close to a polynomial $F(X) \in \mathbb{F}[X]_{<T}$ and $F(\alpha) = z$.

3 Bivariate FRI-based PCS

We start by introducing an abstract interface which we denote as **DFRI** (short for **D**istributed **FRI**) in Section 3.1. In Section 3.2, we describe the construction of the bivariate FRI-based PCS, define its desired security properties, and provide security proofs for them. Towards the end of the section, we discuss how the security of the PCS implies the security of the interactive argument of knowledge compiled from *Pianist's* PIOP.

3.1 Distributed FRI

DFRI (Distributed FRI) is an important building block of our bivariate FRI-based polynomial commitment scheme. The role of **DFRI** in our PCS is similar to the role of **FRI** in the univariate FRI-based PCS (Section 2.4). Instead of proving that a single function $f : D \rightarrow \mathbb{F}$ is δ -close to the Reed-Solomon code, **DFRI** proves that a collection of functions $f_i : D \rightarrow \mathbb{F}$ are all δ -close to the Reed-Solomon code by utilizing a cluster of machines. A basic instantiation of the **DFRI** interface is Parallel FRI, where the machines run the FRI low-degree test for f_i 's in parallel. However, this method produces large proofs whose size grows linearly with the number of machines. In Section 4, we analyze two other **DFRI** instantiations besides Parallel FRI that achieve smaller proof sizes.

The motivation behind defining **DFRI** as an abstract interface is to allow different instantiations and modularize the security proofs for our PCS. For example, we prove in Proposition 1 that the knowledge soundness of our PCS reduces to the knowledge soundness of **DFRI**. This means, to prove that our PCS with a specific **DFRI** instantiation is knowledge sound, we only need to prove that the **DFRI** instantiation is knowledge sound using the following definition:

Definition 7 (Distributed FRI (DFRI)). The *Distributed FRI* interface consists of the following probabilistic polynomial-time interactive algorithms:

- **Commit** $((f_i)_{i=0}^{M-1}, \mathbf{pp}) \rightarrow ((\mathcal{C}_i)_{i=0}^{M-1}, \mathbf{st})$: given public parameters $\mathbf{pp}$ and functions $f_i : D \rightarrow \mathbb{F}$, output the Merkle commitments $\mathcal{C}_i = \text{MT.Commit}(\text{RS.Decode}(f_i))$ and a state $\mathbf{st}$.
- **Prove** $(\alpha, (z_i)_{i=0}^{M-1}, (f_i)_{i=0}^{M-1}, \mathbf{st}, \mathbf{pp}) \rightarrow \pi$: given public parameters $\mathbf{pp}$, functions $f_i : D \rightarrow \mathbb{F}$, field elements $(z_i)_{i=0}^{M-1}$, α and state $\mathbf{st}$, output a proof string π .
- **Verify** $((\mathcal{C}_i)_{i=0}^{M-1}, (z_i)_{i=0}^{M-1}, \alpha, \pi, \mathbf{pp}) \rightarrow 0 \text{ or } 1$: given public parameters $\mathbf{pp}$, commitments $(\mathcal{C}_i)_{i=0}^{M-1}$, field elements $(z_i)_{i=0}^{M-1}$, α and a proof string π , output 0 or 1.

such that the pair of algorithms $\Pi_{\text{DFRI}} = (\text{Prove}, \text{Verify})$ is a public-coin interactive argument for the following relation satisfying **completeness** and **knowledge soundness** as defined in Definition 3:

$$\mathcal{R}_{\text{DFRI}} = \left\{ \left(\begin{array}{l} \mathbf{x} = ((\mathcal{C}_i)_{i=0}^{M-1}, (z_i)_{i=0}^{M-1}, \alpha) \\ \mathbf{w} = (f_0, \dots, f_{M-1}) \end{array} \right) \mid \begin{array}{l} \text{Each } \frac{f_i(X) - z_i}{X - \alpha} \text{ is } \delta\text{-close to RS} \wedge \\ ((\mathcal{C}_i)_{i=0}^{M-1}, \mathbf{st}) = \\ \text{DFRI.Commit}((f_i)_{i=0}^{M-1}, \mathbf{pp}) \\ \text{for some state } \mathbf{st} \end{array} \right\}$$

Note that the relation $\mathcal{R}_{\text{DFRI}} = \mathcal{R}_{\text{DFRI}, \mathbf{pp}}$ is parametrized by the public parameters $\mathbf{pp} = (\mathbb{F}, T, \rho, \delta)$. We will suppress the public parameters from the notation and always write $\mathcal{R}_{\text{DFRI}}$ instead. We briefly explain the intuition for $\mathcal{R}_{\text{DFRI}}$. For an honest prover, f_i 's are the witness polynomials held by the worker nodes, $\mathcal{C}_i$'s are their corresponding commitments, and z_i 's are their evaluations at α . During **DFRI**, the prover proves to the verifier that each function $\frac{f_i(X) - z_i}{X - \alpha}$ is δ -close to **RS** and each $\mathcal{C}_i$ is the commitment of f_i . This implies all f_i 's are δ -close to **RS** and $f_i(\alpha) = z_i$.

3.2 Bivariate FRI-based PCS

3.2.1 Construction

Let $\omega_Y \in \mathbb{F}$ be a primitive M -th root of unity. Let $R_i(Y) \in \mathbb{F}[Y]_{<M}$ be the i -th Lagrange polynomial defined at the M -th roots of unity $\mu_M = \{1, \omega_Y, \dots, \omega_Y^{M-1}\}$. Let

$S_i(Y) \in \mathbb{F}[Y]_{<M-1}$ be the i -th Lagrange polynomial defined at $\mu_M \setminus \{\omega_Y^{M-1}\}$. Let $\mathcal{F}$ be the collection of bivariate polynomials $F(X, Y) \in \mathbb{F}[X, Y]$ of the form

$$F(X, Y) = \sum_{i=0}^{M-1} F_i(X) R_i(Y),$$

where $F_i(X) \in \mathbb{F}[X]_{<T}$. The bivariate FRI-based polynomial commitment scheme **PCS** for $\mathcal{F}$ consists of the following algorithms:

- **Setup**(1^λ) $\rightarrow$ **pp**: Outputs the public parameters **pp** = $(\mathbb{F}, T, l, \rho, \delta)$ containing the field $\mathbb{F}$, the degree bound T , the number of queries l , the code rate ρ , and the proximity parameter δ for **DFRI**.
- **Precomputation**($M, \mathbf{pp}$) $\rightarrow (s_i)_{i=0}^{M-2}$: The verifier computes and outputs $s_i := S_i(\omega_Y^{M-1})$ for $i = 0, \dots, M-2$. This precomputation only depends on the number of machines M and the public parameters **pp**. Hence, its output can be reused for different circuits.
- **Commit**($F, \mathbf{pp}$) $\rightarrow (\mathcal{C}, \mathbf{st})$: The prover computes

$$(\mathcal{C}, \mathbf{st}) := \text{DFRI.Commit}((F_i)_{i=0}^{M-1}, \mathbf{pp})$$

and outputs $(\mathcal{C}, \mathbf{st})$.

- **VerifyPoly**($\mathcal{C}, F, \mathbf{pp}$) $\rightarrow$ 0 or 1: Check if $F \in \mathcal{F}$ and **Commit**($F, \mathbf{pp}$) = $(\mathcal{C}, \mathbf{st})$ for some **st**. Output 1 if both checks pass. Otherwise, output 0.
- **Open**($F, \alpha, \beta, \mathbf{st}, \mathbf{pp}$) $\rightarrow (z, \pi)$: Here we assume $\beta \notin \mu_M$. If $\beta \in \mu_M$, then the protocol is aborted. Since $|\mu_M| = M \ll |\mathbb{F}|$, the probability of this is very small.

1. The prover computes $z_i := F_i(\alpha)$ for all i and $z := F(\alpha, \beta)$.
2. The prover produces a proof π' by running

$$\text{DFRI.Prove}(\alpha, (z_i)_{i=0}^{M-1}, (F_i)_{i=0}^{M-1}, \mathbf{st}, \mathbf{pp}).$$

3. The prover outputs (z, π) where $\pi := ((z_i)_{i=0}^{M-1}, \pi')$.

- **Verify**($\mathcal{C}, \alpha, \beta, z, \pi, \mathbf{pp}$) $\rightarrow$ 0 or 1:

1. The verifier parses $\mathcal{C}$ as $(\mathcal{C}_i)_{i=0}^{M-1}$, parses π as $((z_i)_{i=0}^{M-1}, \pi')$, runs

$$\text{DFRI.Verify}((\mathcal{C}_i)_{i=0}^{M-1}, (z_i)_{i=0}^{M-1}, \alpha, \pi', \mathbf{pp}),$$

and outputs 0 if the verification fails.

2. The verifier computes $h_i := \frac{z_i - z}{\omega_Y^i - \beta}$ for $i = 0, \dots, M-1$.
3. The verifier checks that

$$\sum_{i=0}^{M-2} h_i \cdot s_i = h_{M-1} \tag{1}$$

using the precomputed values $s_i := S_i(\omega_Y^{M-1})$, and outputs 1 if the check passes.

We explain the intuition behind the construction for **Open** and **Verify**. If $F_0, \dots, F_{M-1}$ are the functions committed to by the honest prover in the **Commit** phase (hence, all F_i 's are indeed δ -close to **RS**), then the bivariate polynomial committed to by the prover is $F(X, Y) = \sum_{i=0}^{M-1} F_i(X) R_i(Y)$. Essentially, the prover proves to the verifier that $F(\alpha, \beta) = z$ in two steps:

1. Prove that $F_i(\alpha) = z_i$ for all i .
2. Prove that $F(\alpha, Y) - z$ has a root at $Y = \beta$.

To prove the first step, the prover runs **DFRI.Prove** on polynomials $\{F_i\}_i$, evaluation point α and claimed evaluations $\{z_i\}_i$. By the same reasoning for the univariate FRI-based PCS, the verifier should be convinced that $F_i(\alpha) = z_i$. At this point, the verifier can already construct $F(\alpha, Y)$ by computing $F(\alpha, Y) = \sum_{i=0}^{M-1} F_i(\alpha) R_i(Y) = \sum_{i=0}^{M-1} z_i R_i(Y)$. Directly checking that $F(\alpha, Y) - z$ vanishes at $Y = \beta$ would take time $O(M^2)$. We use an approach that takes time linear in M with a small constant by using the following fact: $F(\alpha, Y) - z$ has a root at $Y = \beta$ if and only if $F(\alpha, Y) - z = H(Y)(Y - \beta)$ for some polynomial $H(Y)$. Notice that the verifier already has all the information it needs to construct H . Since $H(Y) = \frac{F(\alpha, Y) - z}{Y - \beta}$ has degree $M - 2$, it is determined by its evaluation at $M - 1$ points. A convenient choice would be $M - 1$ points chosen from $\mu_M = \{1, \omega_Y, \omega_Y^2, \dots, \omega_Y^{M-1}\}$, because the verifier already knows how to evaluate $F(\alpha, Y)$ at any of these points: evaluating at ω_Y^i simply produces z_i . This is exactly what the verifier does in Step 2 of **Verify**. It constructs $H(Y)$ by computing its evaluation at $1, \omega_Y, \omega_Y^2, \dots, \omega_Y^{M-2}$. To conclude that $F(\alpha, Y) - z = H(Y)(Y - \beta)$, it suffices to show that both sides are equal when evaluated at M distinct points. The verifier already knows that both sides are equal when evaluated at $1, \omega_Y, \omega_Y^2, \dots, \omega_Y^{M-2}$ by the construction of H . It remains to check the equality when $Y = \omega_Y^{M-1}$. This is done in Step 3 of **Verify**.

Handling invalid verifier challenges in implementations. In an implementation of the PCS, the verifier challenge β is derived using the Fiat-Shamir transform. One can use the following technique to avoid getting a β in the invalid set μ_M : (1) Fix a length s , and append a binary string of length s representing 0 to the input of the hash function used to derive β . (2) Check if the resulting challenge β lands inside μ_M . (3) If $\beta \in \mu_M$, increment the binary string appended and go back to step (2). If we have exhausted all possible binary strings of length s , then exit the prover program with an error message.

3.2.2 Security Properties

In this section, we define the security properties for a polynomial commitment scheme PCS.

Completeness. PCS has *completeness* if for any polynomial $F \in \mathcal{F}$ and field elements $\alpha, \beta \in \mathbb{F}$, the following probability is at least $1 - \text{negl}(\lambda)$:

$$\Pr \left[\begin{array}{l} \mathbf{pp} \leftarrow \text{Setup}(1^\lambda) \\ (s_i)_{i=0}^{M-2} \leftarrow \text{Precomputation}(M, \mathbf{pp}) \\ (\mathcal{C}, \mathbf{st}) \leftarrow \text{Commit}(F, \mathbf{pp}) \\ (z, \pi) \leftarrow \text{Open}(F, \alpha, \beta, \mathbf{st}, \mathbf{pp}) \end{array} : \text{Verify}(\mathcal{C}, \alpha, \beta, z, \pi, \mathbf{pp}) = 1 \right]$$

Informally, if the prover followed the protocol to commit and open F , then the verifier always accepts except with negligible probability.

Computationally Polynomial Binding. PCS is *computationally polynomial binding* if for any polynomial-time adversary $\mathcal{A}$ and any tuple $(\mathcal{C}, F, G)$ output by $\mathcal{A}$ given properly generated public parameters $\mathbf{pp}$, we have

$$\Pr \left[\begin{array}{l} \text{VerifyPoly}(\mathcal{C}, F, \mathbf{pp}) = 1, \\ \text{VerifyPoly}(\mathcal{C}, G, \mathbf{pp}) = 1, \\ F \neq G \end{array} \right] = \text{negl}(\lambda).$$

This property ensures that, with high probability, a commitment cannot be successfully opened to two different polynomials by a polynomial-time prover.

Computationally Evaluation Binding. PCS is *computationally evaluation binding* if for any polynomial-time adversary $\mathcal{A}$, any tuple $(\mathcal{C}, \alpha, \beta, (z, \pi), (z', \pi'))$ output by $\mathcal{A}$ given properly generated public parameters $\mathbf{pp}$, we have

$$\Pr \left[\begin{array}{c} \text{Verify}(\mathcal{C}, \alpha, \beta, z, \pi, \mathbf{pp}) = 1, \\ \text{Verify}(\mathcal{C}, \alpha, \beta, z', \pi', \mathbf{pp}) = 1, \\ z \neq z' \end{array} \right] = \text{negl}(\lambda).$$

Knowledge Soundness. PCS has *knowledge soundness* if any probabilistic polynomial-time prover $\mathcal{P}^*$ who successfully convinces the verifier the validity of an opening $(\mathcal{C}, \alpha, \beta, z)$ also *knows* a polynomial $F(X, Y)$ that explains this opening, i.e. $\mathcal{C}$ is the commitment of F and $F(\alpha, \beta) = z$.

Formally, we view the pair of algorithms $\Pi_{\text{PCS}} = (\mathcal{P} = \text{Open}, \mathcal{V} = \text{Verify})$ as a public-coin interactive argument for the following relation:

$$\mathcal{R}_{\text{PCS}} := \left\{ \left(\begin{array}{c} \mathbb{X} = (\mathcal{C}, \alpha, \beta, z) \\ \mathbb{W} = (f_0, \dots, f_{M-1}) \end{array} \right) \mid \begin{array}{l} \text{Each } f_i : D \rightarrow \mathbb{F} \text{ is } \delta\text{-close to } \mathbf{RS} \wedge \\ ((\mathcal{C}_i)_{i=0}^{M-1}, \mathbf{st}) = \text{DFRI.Commit}((f_i)_{i=0}^{M-1}, \mathbf{pp}) \\ \text{for some state } \mathbf{st} \wedge \\ \sum_i F_i(\alpha) R_i(\beta) = z \\ \text{where } F_i(X) = \mathbf{RS.Decode}(f_i) \end{array} \right\}$$

The polynomial commitment scheme PCS has **knowledge soundness** if Π_{PCS} has knowledge soundness as an interactive argument.

3.2.3 Security Analysis

In this section, we prove that the polynomial commitment scheme PCS defined in Section 3.2.1 has all the security properties defined in Section 3.2.2 when using a valid instantiation of DFRI, i.e., when Π_{DFRI} is an interactive argument of knowledge. Then, we prove that this is enough to establish the security of the interactive protocol obtained by compiling *Pianist's* bivariate polynomial IOP with our PCS.

Security of the polynomial commitment scheme. If Π_{DFRI} has completeness, then PCS also has completeness. This follows directly from the construction of PCS. PCS also satisfies polynomial binding and evaluation binding. Polynomial binding follows from the fact that two polynomials of degree $< T$ cannot agree on every point in the evaluation domain D of size $\rho^{-1}T > T$ and the tree-collision property of Merkle trees. We prove these three security properties later in this section.

We will prove that PCS has knowledge soundness if Π_{DFRI} has knowledge soundness after proving a useful lemma:

Lemma 1. Suppose that for functions $f_i : D \rightarrow \mathbb{F}$, elements $\alpha, \beta, z_i, z \in \mathbb{F}$ and proof π' , the following conditions are satisfied:

1. $\frac{f_i(X) - z_i}{X - \alpha}$ is δ -close to $\mathbf{RS}$ for $i = 0, \dots, M-1$.
2. $\text{PCS.Verify}(\mathcal{C}, \alpha, \beta, z, ((z_i)_{i=0}^{M-1}, \pi'), \mathbf{pp}) = 1$.

Let $F_i(X) := \mathbf{RS.Decode}(f_i)$ and $F(X, Y) := \sum_{i=0}^{M-1} F_i(X) R_i(Y)$. Then, $F_i(\alpha) = z_i$ for all i and $F(\alpha, \beta) = z$.

Proof. Since $\frac{f_i(X) - z_i}{X - \alpha}$ is δ -close to $\mathbf{RS}$, there is a polynomial $P_i(X) \in \mathbb{F}[X]_{<T}$ that is δ -close to it. This means $f_i(X)$ is δ -close to the polynomial $G_i(X) := P_i(X)(X - \alpha) + z_i$ for each i . It is clear that $G_i(\alpha) = z_i$ for all i .

Let $h_i := \frac{z_i - z}{\omega_Y^i - \beta}$ for $i = 0, \dots, M-1$, $H(Y) := \sum_{i=0}^{M-2} h_i \cdot S_i(Y)$, and $G(X, Y) := \sum_{i=0}^{M-1} G_i(X) R_i(Y)$. Then $G(\alpha, \omega_Y^i) = G_i(\alpha) = z_i$ for $i = 0, \dots, M-1$. Therefore,

$$G(\alpha, \omega_Y^i) - z - H(\omega_Y^i)(\omega_Y^i - \beta) = z_i - z - h_i \cdot (\omega_Y^i - \beta) = 0,$$

for $i = 0, \dots, M-2$. Since Equation 1 holds, we also have

$$\begin{aligned} & G(\alpha, \omega_Y^{M-1}) - z - H(\omega_Y^{M-1})(\omega_Y^{M-1} - \beta) \\ &= z_{M-1} - z - \sum_{i=0}^{M-2} h_i \cdot S_i(\omega_Y^{M-1})(\omega_Y^{M-1} - \beta) \\ &= \left(\omega_Y^{M-1} - \beta \right) \left(h_{M-1} - \sum_{i=0}^{M-2} h_i \cdot S_i(\omega_Y^{M-1}) \right) = 0. \end{aligned}$$

We have shown that the polynomial $G(\alpha, Y) - z - H(Y)(Y - \beta) \in \mathbb{F}[Y]$ of degree $< M$ has M distinct roots, therefore it must be the zero polynomial. Taking $Y = \beta$, we get $G(\alpha, \beta) = z$. Since we are in the unique decoding regime, there is only one polynomial of degree $< T$ in the ball of radius δ around $f_i(X)$. Thus, $G_i(X) = \text{RS.Decode}(f_i) = F_i(X)$, which means $F(\alpha, \beta) = G(\alpha, \beta) = z$. $\square$

Proposition 1. *If Π_{DFRI} has knowledge soundness, then PCS also has knowledge soundness.*

Proof. We describe $\mathcal{E}_{\text{PCS}}$, the extractor for the polynomial commitment scheme. Let $\mathcal{P}_{\text{PCS}}^*$ denote the PCS prover. Since in PCS, the prover does not send any message before running DFRI.Prove , we can set $\mathcal{P}_{\text{DFRI}}^* := \mathcal{P}_{\text{PCS}}^*$. Since $\mathcal{E}_{\text{PCS}}$ has access to $\mathcal{P}_{\text{DFRI}}^*$'s messages, it can run $\mathcal{E}_{\text{DFRI}}$ and output the witness $(f_0, \dots, f_{M-1})$ for $\mathcal{R}_{\text{DFRI}}$.

Let A be the event that $\mathcal{V}_{\text{DFRI}}$ accepts, and

$$(\mathbb{X} = (\mathcal{C}, (z_i)_{i=0}^{M-1}, \alpha), \mathbb{W} = (f_0, \dots, f_{M-1})) \notin \mathcal{R}_{\text{DFRI}}.$$

Let B be the event that $\mathcal{V}_{\text{PCS}}$ accepts, and

$$(\mathbb{X} = (\mathcal{C}, \alpha, \beta, z), \mathbb{W} = (f_0, \dots, f_{M-1})) \notin \mathcal{R}_{\text{PCS}}.$$

By the knowledge soundness of Π_{DFRI} , $\Pr[A] = \text{negl}(\lambda)$. To show knowledge soundness of PCS, we need to show $\Pr[B] = \text{negl}(\lambda)$. We have $\Pr[B] = \Pr[A \cap B] + \Pr[A^c \cap B]$, and $\Pr[A \cap B] = \text{negl}(\lambda)$. We will show that $\Pr[A^c \cap B] = 0$. In the event $A^c \cap B$, since B happened, we have that $\mathcal{V}_{\text{PCS}}$ accepts. This means $\mathcal{V}_{\text{DFRI}}$ accepted as well. Since A did not happen, we must have $((\mathcal{C}, (z_i)_{i=0}^{M-1}, \alpha), (f_0, \dots, f_{M-1})) \in \mathcal{R}_{\text{DFRI}}$. Explicitly, this means: (1) each $\frac{f_i(X) - z_i}{X - \alpha}$ is δ -close to RS, therefore each f_i is δ -close to RS, and (2) $(\mathcal{C}, \text{st}) = \text{DFRI.Commit}((f_i)_{i=0}^{M-1}, \text{pp})$ for some state st . By Lemma 1, $((\mathcal{C}, \alpha, \beta, z), (f_0, \dots, f_{M-1})) \in \mathcal{R}_{\text{PCS}}$, contradicting B . $\square$

Proposition 2. *PCS is complete if Π_{DFRI} is complete.*

Proof. Let F be a bivariate polynomial of the form

$$F(X, Y) = \sum_{i=0}^{M-1} F_i(X) R_i(Y)$$

with $\deg F_i < T$ and let $(\alpha, \beta) \in \mathbb{F}^2$ with $\beta \notin \mu_M$. Suppose we have

$$\begin{aligned} \text{pp} &\leftarrow \text{Setup}(1^\lambda), \\ (s_i)_{i=0}^{M-2} &\leftarrow \text{Precomputation}(M, \text{pp}), \\ (\mathcal{C}, \text{st}) &\leftarrow \text{Commit}(F, \text{pp}), \\ (z, \pi) &\leftarrow \text{Open}(F, \alpha, \beta, \text{st}, \text{pp}). \end{aligned}$$

Since $z_i = F_i(\alpha)$, each $\frac{F_i(X) - z_i}{X - \alpha}$ is a polynomial of degree $< T - 1$. By completeness of Π_{DFRI} , the first check in **Verify** will pass.

Since $F(\alpha, \beta) = z$, $Y - \beta \mid F(\alpha, Y) - z$. Therefore, $\tilde{H}(Y) := \frac{F(\alpha, Y) - z}{Y - \beta}$ is a polynomial in $\mathbb{F}[Y]$ of degree $< M - 1$, with

$$\tilde{H}(\omega_Y^i) = \frac{F(\alpha, \omega_Y^i) - z}{\omega_Y^i - \beta} = \frac{z_i - z}{\omega_Y^i - \beta} = h_i$$

for $i = 0, \dots, M - 1$. Consider the polynomial

$$H(Y) := \sum_{i=0}^{M-2} h_i \cdot S_i(Y)$$

of degree $< M - 1$. Clearly, $H(\omega_Y^i) = h_i$ for $i = 0, \dots, M - 2$. $\tilde{H}$ and H are two polynomials of degree $< M - 1$ and share the same evaluations at $M - 1$ points, so must be equal. This means the verifier's check using Equation 1 passes:

$$\sum_{i=0}^{M-2} h_i \cdot s_i = \sum_{i=0}^{M-2} h_i \cdot S_i(\omega_Y^{M-1}) = H(\omega_Y^{M-1}) = \tilde{H}(\omega_Y^{M-1}) = h_{M-1}$$

If the opening point (α, β) sampled by the verifier happens to be one such that $\beta \in \mu_M$, then the protocol aborts. This happens with probability $\frac{M}{|\mathbb{F}|}$ which is $\text{negl}(\lambda)$ because $\mathbb{F}$ has size at least super-polynomial in λ . $\square$

Proposition 3. *PCS satisfies polynomial binding.*

Proof. Suppose $\mathcal{A}$ is an efficient adversary that can output a tuple $(\mathcal{C}, F, G)$ such that $\text{VerifyPoly}(\mathcal{C}, F, \mathbf{pp}) = 1$ and $\text{VerifyPoly}(\mathcal{C}, G, \mathbf{pp}) = 1$ and $F \neq G$ with non-negligible probability. For any such tuple, let $\mathcal{C}_0, \dots, \mathcal{C}_{M-1}$ be the Merkle commitments in $\mathcal{C}$. Since both openings successfully verify, we must have $F, G \in \mathcal{F}$ and $\text{MT.Commit}(F_i) = \mathcal{C}_i = \text{MT.Commit}(G_i)$ for all i . Since $F_i \neq G_i$ for some i , $((F_i(x))_{x \in D}) \neq ((G_i(x))_{x \in D})$ as vectors. Thus, there is an efficient algorithm $\mathcal{B}$ using $\mathcal{A}$ as a subroutine to output a hash collision for the collision-resistant hash function $\mathcal{H}$ with non-negligible probability. This is a contradiction. $\square$

Proposition 4. *PCS satisfies evaluation binding.*

Proof. Suppose we have two sets of inputs satisfying

$$\text{Verify}(\mathcal{C}, \alpha, \beta, z, \pi, \mathbf{pp}) = 1 \wedge \text{Verify}(\mathcal{C}, \alpha, \beta, w, \gamma, \mathbf{pp}) = 1$$

Let $((z_i)_{i=0}^{M-1}, \pi')$ be the result of parsing π . Let $((w_i)_{i=0}^{M-1}, \gamma')$ be the result of parsing γ . Since $\text{DFRI.Verify}(\mathcal{C}, (z_i)_{i=0}^{M-1}, \alpha, \pi', \mathbf{pp}) = 1$, with high probability $q_i(X) = \frac{f_i(X) - z_i}{X - \alpha}$ are δ -close to RS, where f_i 's are the functions committed in $\mathcal{C}$. Let $F_i(X) := \text{RS.Decode}(f_i)$ and let $F(X, Y) := \sum_{i=0}^{M-1} F_i(X) R_i(Y)$. By Lemma 1, we have $F(\alpha, \beta) = z$. Similarly, since $\text{DFRI.Verify}(\mathcal{C}, (w_i)_{i=0}^{M-1}, \alpha, \gamma', \mathbf{pp}) = 1$, with high probability $q'_i(X) = \frac{f_i(X) - w_i}{X - \alpha}$ are δ -close to RS. By Lemma 1 again, $F(\alpha, \beta) = w$. This shows $z = w$. $\square$

Security of the compiled protocol. In *Pianist* [LXZ⁺24], they built an interactive argument of knowledge ([LXZ⁺24], Protocol 3) by compiling the bivariate polynomial IOP ([LXZ⁺24], Protocol 1) with the DKZG polynomial commitment scheme ([LXZ⁺24], Protocol 2). The security of the final interactive protocol comes from Theorem 4 of [BFS20], which says if the polynomial commitment scheme Γ has witness-extended emulation and the polynomial IOP for the relation $\mathcal{R}$ has negligible knowledge error, then the compiled

protocol is a public-coin interactive argument for $\mathcal{R}$ with witness-extended emulation. It was proven ([Lin01], Lemma 3.1) that any interactive argument of knowledge has witness-extended emulation. Therefore, it suffices to show that the polynomial commitment scheme Γ has knowledge soundness. This is exactly what is done in *Pianist*. In this work, we would like to compile *Pianist*'s bivariate polynomial IOP with our bivariate FRI-based polynomial commitment scheme.

Proposition 5. *Let Π_{compiled} be the interactive protocol compiled from *Pianist*'s bivariate PIOP using the bivariate FRI-based polynomial commitment scheme PCS. If the distributed FRI subprotocol Π_{DFRI} used in PCS is an interactive argument for $\mathcal{R}_{\text{DFRI}}$ with negligible knowledge error, then Π_{compiled} is a public-coin interactive argument for general circuits with witness-extended emulation.*

Proof. We have proven that if Π_{DFRI} is an interactive argument with completeness and knowledge soundness, then the polynomial commitment scheme PCS has completeness (Proposition 2) and knowledge soundness (Proposition 1). Since *Pianist*'s bivariate polynomial IOP has negligible knowledge error ([LXZ⁺24], Theorem 1), the conclusion follows from Theorem 4 of [BFS20]. $\square$

4 Instantiations for DFRI

In this section, we start by describing three methods for instantiating DFRI: Parallel FRI, Distributed Batched FRI, and Fold-and-Batch. Then, in Section 4.1, we compare their asymptotic costs, and in Section 4.2, we discuss their security.

We briefly review the notations that will be used in the description of the DFRI methods below. Let N be the size of the input circuit, M be the number of worker nodes, and let $T = \frac{N}{M}$ be the size of the sub-circuit handled by each worker node. $\mathcal{P}_i$ denotes the i -th worker node. D is the evaluation domain for the worker nodes' sub-circuits, and has size $|D| = \rho^{-1}T$ where ρ^{-1} is the FRI-blowup factor.

In Definition 7, the DFRI.Prove algorithm takes as input functions $\{F_i\}_i$, an evaluation point α and claimed evaluations $\{z_i\}_i$. This definition is convenient for describing our bivariate PCS, since the PCS not only needs to prove that the input functions $\{F_i\}_i$ are low-degree, but also needs to prove their evaluations at α . In this section, our focus is on how the prover runs the FRI low-degree test in the distributed setting. To simplify notations, we will present all DFRI algorithms and their security proofs as if the low-degree tests were run directly on $\{F_i\}_i$. It is straightforward to see that all the DFRI instantiations and their security proofs can be converted to the complete version by changing each $F_i(X)$ to $Q_i(X) := \frac{F_i(X) - z_i}{X - \alpha}$, and have the verifier compute the oracle query answers for Q_i 's from those for F_i 's.

Parallel FRI. The simplest method to run DFRI is to let each worker node $\mathcal{P}_i$ run a separate instance of FRI on the input function $F_i : D \rightarrow \mathbb{F}$. Then, the verifier checks each one of the M FRI proofs. In the non-interactive version of the protocol, since each worker node derives its folding challenges using the Fiat-Shamir transformation, no communication is needed. Therefore, we do not need a master node in this setting. The overall runtime for Parallel FRI is $O(T)$, the runtime for FRI with input size T . The proof size is $O(M \log^2 T)$.

Distributed Batched FRI. In [Che24], the author described a distributed protocol for running batched FRI. For convenience, we will refer to this method as *Distributed Batched FRI*. We provide a complete description of this protocol adapted to the setting of DFRI in Protocol 1. In [Che24], each prover node operates on multiple polynomials. In our distributed setting, each prover node operates on a single polynomial. Furthermore,

we assume that a prover node already has the evaluation vector of its polynomial at the beginning of the protocol. This is because the bivariate FRI-based PCS (Section 3.2.1) requires the computation of evaluation vectors at the **Commit** phase. Since **DFRI** is invoked at a later stage, each $\mathcal{P}_i$ already has its evaluation vector by then. Due to the absence of FFT computations, the runtime for each worker node becomes $O(T)$ (as opposed to $O(T \log T)$). However, the runtime of $\mathcal{P}_0$ and the total communication remain $O(N)$. The communication bottleneck occurs when $\mathcal{P}_0$ receives an evaluation vector of $\rho^{-1}T$ field elements from each worker node (Protocol 1, Step 3). In Table 6, we provide a step-by-step cost analysis for Distribute Batched FRI.

Protocol 1: Distributed Batched FRI [Che24]

Suppose the prover $\mathcal{P}$ consists of M machines $\mathcal{P}_0, \dots, \mathcal{P}_{M-1}$ where $\mathcal{P}_0$ is the master node. Each $\mathcal{P}_i$ holds the evaluation vector $(F_i(x))_{x \in D}$ for its local witness polynomial $F_i(X)$. The protocol proceeds as follows:

1. Each $\mathcal{P}_i$ builds the Merkle tree from its evaluation vector $(F_i(x))_{x \in D}$ and sends the commitment to $\mathcal{P}_0$. $\mathcal{P}_0$ sends all commitments to $\mathcal{V}$.
2. $\mathcal{P}_0$ receives a random challenge θ from $\mathcal{V}$ and sends it to $\mathcal{P}_i$'s.
3. Each $\mathcal{P}_i$ computes the new evaluation vector $(\theta^i F_i(x))_{x \in D}$ and sends it to $\mathcal{P}_0$.
4. $\mathcal{P}_0$ computes the evaluation vector $(F(x))_{x \in D}$ where $F(X) = \sum_i \theta^i F_i(X)$.
5. $\mathcal{P}_0$ and $\mathcal{V}$ run FRI on $F(X)$.
6. Extra consistency checks for batched FRI: For each queried $g \in D$, each $\mathcal{P}_i$ sends $F_i(g)$ and its Merkle authentication path to $\mathcal{P}_0$. $\mathcal{P}_0$ sends all received data to $\mathcal{V}$. $\mathcal{V}$ checks that the Merkle authentication paths are valid and $F(g) = \sum_{i=0}^{M-1} \theta^i F_i(g)$.

Fold-and-Batch. Fold-and-Batch is a **DFRI** instantiation that improves the communication cost of Distributed Batched FRI [Che24]. The idea is to make $\mathcal{P}_i$ fold its local polynomial to a smaller size before sending it to $\mathcal{P}_0$. This reduces $\mathcal{P}_0$'s runtime and the overall communication. However, it slightly increases the proof size and verification time.

Fold-and-Batch can be viewed as an “interpolation” between Distributed Batched FRI and Parallel FRI. A folding parameter $K \in [1, T]$ (K a power of 2) is used to adjust how much folding is done locally by the worker nodes versus how much folding is done by the master node using batched FRI. When $K = T$, Fold-and-Batch is the same as Distributed Batched FRI. When $K = 1$, Fold-and-Batch is essentially Parallel FRI (with adjustments). By setting $K = \frac{T}{M}$, Fold-and-Batch can achieve a runtime of $O(T + M \log M \log T)$ for all provers and the same for total communication, while Distributed Batched FRI costs $O(N)$ for both. Below is a description of the Fold-and-Batch protocol:

Protocol 2: Fold-and-Batch

Suppose the prover $\mathcal{P}$ consists of M machines $\mathcal{P}_0, \dots, \mathcal{P}_{M-1}$ where $\mathcal{P}_0$ is the master node. Each $\mathcal{P}_i$ holds the evaluation vector $(F_i(x))_{x \in D}$ for its local witness polynomial $F_i(X)$. Let K be the Fold-and-Batch folding parameter. The protocol proceeds as follows:

1. Each $\mathcal{P}_i$ builds the Merkle tree from its evaluation vector $(F_i(x))_{x \in D}$ and sends the commitment to $\mathcal{P}_0$. $\mathcal{P}_0$ sends all commitments to $\mathcal{V}$.
2. Each $\mathcal{P}_i$ folds its evaluation vector $(F_i(x))_{x \in D}$ for $\log \frac{T}{K}$ times. The result is a

- vector $(G_i(y))_{y \in D'}$ where $G_i(X) = F_i^{(\log \frac{T}{K})}(X)$ is a polynomial of degree $< K$ and $D' = \mu_{\rho^{-1}K}$.
3. Each $\mathcal{P}_i$ sends $\mathcal{P}_0$ the Merkle commitments for every intermediate folded polynomial. This includes the Merkle commitments of $(G_i(y))_{y \in D'}$. $\mathcal{P}_0$ sends these commitments to $\mathcal{V}$.
 4. $\mathcal{P}_0$ receives a random challenge θ from $\mathcal{V}$ and sends it to $\mathcal{P}_i$'s.
 5. Each $\mathcal{P}_i$ computes the vector $(\theta^i G_i(y))_{y \in D'}$ and sends it to $\mathcal{P}_0$.
 6. $\mathcal{P}_0$ computes the vector $(G(y))_{y \in D'}$ where $G(X) = \sum_{i=0}^{M-1} \theta^i G_i(X)$ has degree $< K$.
 7. $\mathcal{P}_0$ and $\mathcal{V}$ perform the FRI folding phase for $G(X)$.
 8. During the query phase, for each query point $g \in D = \mu_{\rho^{-1}T}$, $\mathcal{P}_0$ sends it to all $\mathcal{P}_i$'s. Each $\mathcal{P}_i$ sends back $F_i^{(0)}(g)$, $F_i^{(0)}(-g)$, $F_i^{(1)}(g^2)$, $F_i^{(1)}(-g^2)$, $\dots$, $F_i^{(\log \frac{T}{K})}(g^{\frac{T}{K}})$, $F_i^{(\log \frac{T}{K})}(-g^{\frac{T}{K}})$ and their Merkle authentication paths which $\mathcal{P}_0$ sends to $\mathcal{V}$. (This is to check that each $\mathcal{P}_i$ folded $F_i(X)$ correctly)
 9. For each query g , let $h := g^{\frac{T}{K}}$. $\mathcal{P}_0$ in addition sends the values $G^{(0)}(h)$, $G^{(0)}(-h)$, $G^{(1)}(h^2)$, $G^{(1)}(-h^2)$, $\dots$, $G^{\log K}(h^K)$ and their Merkle authentication paths to $\mathcal{V}$. (This is to check that $\mathcal{P}_0$ folded $G(X)$ correctly)
 10. For each g and the corresponding $h = g^{\frac{T}{K}}$, $\mathcal{V}$ checks that:
 - (a) Each $\mathcal{P}_i$ folded $F_i(X)$ correctly using Equation 2.
 - (b) $\mathcal{P}_0$ folded $G(X)$ correctly using Equation 2.
 - (c) $G(h) = \sum_{i=0}^{M-1} \theta^i G_i(h)$. $\mathcal{V}$ indeed has all the necessary values to perform this check, since $G(h) = G^{(0)}(h)$ and $G_i(h) = F_i^{(\log \frac{T}{K})}(h) = F_i^{(\log \frac{T}{K})}(g^{\frac{T}{K}})$ were sent to $\mathcal{V}$ during Step 8 and 9.

4.1 Asymptotic Costs

Table 2 shows the asymptotic complexities of the three DFRI methods and compares them with monolithic FRI. The cost of Distributed Batched FRI and Fold-and-Batch is analyzed step by step in Appendix C, Tables 6 and 7.

Table 2: A comparison of different DFRI instantiations

	Monolithic FRI	Distributed Batched FRI	Fold-and-Batch	Parallel FRI
$\mathcal{P}_0$ runtime	$O(N)$	$O(N)$	$O(MK + M \log \frac{T}{K} \log T)$	—
$\mathcal{P}_i$ runtime	—	$O(T)$	$O(T)$	$O(T)$
Total communication between $\mathcal{P}_0$ and $\mathcal{P}_i$	—	$O(N)$	$O(MK + M \log \frac{T}{K} \log T)$	—
Communication for $\mathcal{P}_i$	—	$O(T)$	$O(K + \log \frac{T}{K} \log T)$	—
Proof size / Verifier runtime	$O(\log^2 N)$	$O(\log^2 T + M \log T)$	$O(M \log \frac{T}{K} \log T + \log^2 K)$	$O(M \log^2 T)$

4.2 Security

We now establish the security of the three **DFRI** instantiations: Parallel FRI, Distributed Batched FRI, and Fold-and-Batch. Specifically, we need to prove that these instantiations are interactive arguments of knowledge for $\mathcal{R}_{\text{DFRI}}$. Then, by Proposition 1, the bivariate FRI-based PCS has knowledge soundness when instantiated with these **DFRI** methods.

In [BGK⁺23], the authors proved the knowledge soundness of the non-interactive version of FRI and batched FRI using the following approach: First, prove that the IOP version of FRI and batched FRI have round-by-round knowledge soundness. Then, they compile the IOP into a non-interactive argument using the BCS transformation [BSCS16]. We will follow this approach and argue the security for the three instantiations of **DFRI**.

The BCS transformation turns any IOP into a non-interactive argument in two steps:

1. Whenever the IOP prover sends an oracle string, the interactive argument prover sends a Merkle commitment of that string. Whenever the IOP verifier queries the oracle, the interactive argument prover sends the queried value together with a Merkle opening proof.
2. The protocol is made non-interactive using the Fiat-Shamir transformation [FS86].

In Theorem 3 of [CMS19], the authors proved that if an IOP has round-by-round knowledge soundness, then the BCS-compiled non-interactive argument has knowledge soundness. Furthermore, it has been shown that applying only the first step of the BCS transformation to a round-by-round knowledge sound IOP produces a knowledge sound interactive argument ([CY24], Lemma 26.1.4). The transformation consisting of only the first step of the BCS transformation is called the “iBCS” (*interactive BCS*) transformation in [CY24]. Therefore, to show that a **DFRI** instantiation has knowledge soundness as an interactive argument, it suffices to show that the corresponding IOP has round-by-round knowledge soundness. We summarize the protocol compilation process, starting from any of the three **DFRI** IOPs and ending with a public-coin interactive argument of knowledge for general circuits, in Figure 1 below.

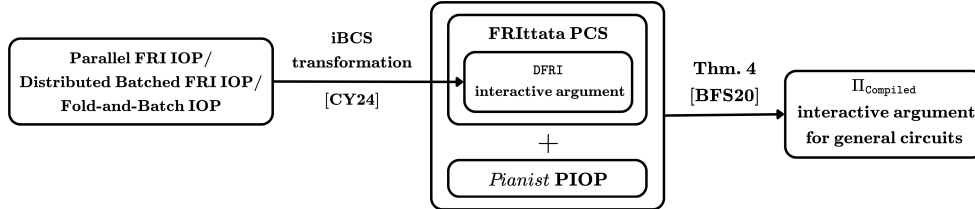


Figure 1: Protocol Compilation Diagram

For Distributed Batched FRI (i.e., Batched FRI), its round-by-round knowledge soundness is shown in [BGK⁺23], Theorem 5.11. The same theorem also proves the round-by-round knowledge soundness of FRI. We will follow this proof and establish the round-by-round knowledge soundness of Parallel FRI and Fold-and-Batch. For completeness, we formally define the parallel execution of multiple IOPs in Appendix D. Then, Parallel FRI can be viewed as the parallel execution of M different instances of FRI.

Notations. We define some additional notations that will be used in the upcoming proofs. Let D be the evaluation domain of FRI, and let l be the number of FRI queries. Let $G_{i,j}$ denote the polynomial obtained by the honest j -th worker node during the i -th round of Parallel FRI or Fold-and-Batch. In other words, $G_{i,j}$ is the polynomial obtained by honestly folding the input polynomial $G_{0,j}$ for i rounds, where the input polynomial itself could be δ -close to $\text{RS}^{(0)}$ or not. Let $G_{i,j}^*$ denote the corresponding polynomial sent

by a potentially malicious prover. We assume $G_{0,j}^* = G_{0,j}$ for all $j \in [M]$. Since we are dealing with IOPs, we simply use the same notation G to denote the oracle string of a polynomial G sent by the prover, with the understanding that the verifier does not have to read it in full. For Parallel FRI and the partial FRI folding phase of Fold-and-Batch, we use $\mathbf{G}_i$ to denote $(G_{i,1}, \dots, G_{i,M})$, $\mathbf{G}_i^*$ to denote $(G_{i,1}^*, \dots, G_{i,M}^*)$, and $\mathbf{c}_i = (c_{i,1}, \dots, c_{i,M})$ to denote the random folding challenges drawn by the verifier during round i . If the verifier only draws a single FRI folding challenge during round i , we denote it c_i . $\mathbf{s}$ denotes the set of FRI query positions. τ_i denotes the transcript of messages up to and including round i .

We assume the Fold-and-Batch IOP $\Pi_{\text{Fold-and-Batch}}$ is run with the following parameters:

- k partial FRI folding rounds, i.e., each worker node folds its local polynomial k times.
- r batched FRI folding rounds, i.e., the batched polynomial becomes a constant after being folded r times by the master node.

The predicate **CorrectFolding**(i) means that all the FRI foldings done during the protocol up to and including the i -th round are honest. **IncorrectFolding**(i) means the prover cheated in at least one FRI folding performed up to and including the i -th round. For Fold-and-Batch, we abuse notations and treat the batched FRI linear combination $F := \sum_{j=1}^M \theta^{j-1} G_j$ as one of the FRI foldings. For example, if the prover sent $F^* \neq F = \sum_{j=1}^M \theta^{j-1} G_j$ during round k , then **IncorrectFolding**(i) = **True** for all $i \geq k$.

4.2.1 Round-by-round Knowledge Soundness of Parallel FRI

To prove the round-by-round knowledge soundness of Parallel FRI, we need to define a doomed set for it. We start by recalling the doomed set for FRI defined in [BGK⁺23]:

Definition 8 ([BGK⁺23], Lemma 5.2, 5.3). The doomed set $\mathcal{D}_{\text{FRI}}$ for FRI with k folding rounds is defined as follows:

1. $(\mathbb{x}, \emptyset) \in \mathcal{D}_{\text{FRI}}$ for all possible $\mathbb{x}$.
2. If $\mathbb{x} \notin \mathcal{L}$, then $(\mathbb{x}, \tau_0) \in \mathcal{D}_{\text{FRI}}$ if and only if G_1 is δ -far from $\text{RS}^{(1)}$.
3. If $\mathbb{x} \notin \mathcal{L}$ and $i \in \{1, \dots, k-1\}$, then $(\mathbb{x}, \tau_i) \in \mathcal{D}_{\text{FRI}} \iff \text{IncorrectFolding}(i)$ or G_{i+1} is δ -far from $\text{RS}^{(i+1)}$.
4. (Query phase) If $\mathbb{x} \notin \mathcal{L}$ then for a complete transcript τ_k , $(\mathbb{x}, \tau_k) \in \mathcal{D}_{\text{FRI}} \iff$ one of the consistency checks performed by the verifier fails.

Using the doomed set for FRI, we can define a doomed set for Parallel FRI and prove its round-by-round knowledge soundness:

Proposition 6. Define the language $\mathcal{L} := \mathcal{L}_{\text{RS}, \delta}^M$ where $\mathcal{L}_{\text{RS}, \delta} = \{w \in \mathbb{F}^D \mid \Delta(w, \text{RS}) < \delta\}$ is the language of words that are δ -close to RS . Assume the Parallel FRI IOP $\Pi_{\text{ParallelFRI}}$ is run with $\delta < \frac{1-\rho}{2}$ (unique decoding radius), then it has round-by-round knowledge error

$$\varepsilon = \max \left\{ \frac{|D|}{|\mathbb{F}|}, (1-\delta)^l \right\}$$

Proof. Define the doomed set $\mathcal{D}$ for $\Pi_{\text{ParallelFRI}}$ as follows: for any input $\mathbb{x} = (\mathbb{x}_1, \dots, \mathbb{x}_M)$ and an i -th round transcript τ_i , $(\mathbb{x}, \tau_i) \in \mathcal{D}$ if and only if $(\mathbb{x}_j, p_j(\tau_i)) \in \mathcal{D}_{\text{FRI}}$ for some $j \in [M]$. We check each property for round-by-round knowledge soundness:

1. For any input $\mathbb{x} = (\mathbb{x}_1, \dots, \mathbb{x}_M)$, by definition $(\mathbb{x}_j, \emptyset) \in \mathcal{D}_{\text{FRI}}$ for all j . Hence, $(\mathbb{x}, \emptyset) \in \mathcal{D}$.

2. For any input $\mathbb{x} = (\mathbb{x}_1, \dots, \mathbb{x}_M)$ and complete transcript τ , $(\mathbb{x}_j, p_j(\tau))$ is a complete transcript for $\mathbb{x}_j$. If $(\mathbb{x}, \tau) \in \mathcal{D}$ then $(\mathbb{x}_j, p_j(\tau)) \in \mathcal{D}_{\text{FRI}}$ for some j . This means $\mathcal{V}_j$ rejects and so does $\mathcal{V}$.
3. We treat the cases $\mathbb{x} \in \mathcal{L}$ and $\mathbb{x} \notin \mathcal{L}$ separately. Suppose $\mathbb{x} \in \mathcal{L}$, then the conditions $(\mathbb{x}, \tau_{i-1}) \in \mathcal{D}$ and $\Pr_{\mathbf{c}_i \leftarrow C_i} [(\mathbb{x}, \tau_{i-1} \parallel m \parallel \mathbf{c}) \notin \mathcal{D}] > \varepsilon$ are both true only when $i = 0$, i.e. when $(\mathbb{x}, \tau_{i-1}) = (\mathbb{x}, \emptyset)$. This is because, by definition, $(\mathbb{x}, \tau) \in \mathcal{D}$ only when $\tau = \emptyset$. In this case, the prover message m is $\mathbf{G}_0$. All $G_{0,j}$'s are δ -close to $\text{RS}^{(0)}$ as $\mathbb{x} \in \mathcal{L}$. Thus, the extractor can simply output the witness $\mathbf{w} = \mathbf{G}_0$.

Suppose $\mathbb{x} \notin \mathcal{L}$. We will show that it is not possible to have $(\mathbb{x}, \tau_{i-1}) \in \mathcal{D}$ and $\Pr_{\mathbf{c}_i \leftarrow C_i} [(\mathbb{x}, \tau_{i-1} \parallel m \parallel \mathbf{c}_i) \notin \mathcal{D}] > \varepsilon$ at the same time:

- Round -1 to Round 0 : Suppose $(\mathbb{x}, \emptyset) \in \mathcal{D}$ and $\Pr_{\mathbf{c}_0 \leftarrow \mathbb{F}} [(\mathbb{x}, \mathbf{G}_0 \parallel \mathbf{c}_0) \notin \mathcal{D}] > \varepsilon$.

Rewriting using the definition of $\mathcal{D}$ we get

$$\Pr_{\mathbf{c}_0 \leftarrow \mathbb{F}} [G_{1,j} \text{ is } \delta\text{-close to } \text{RS}^{(1)} \text{ for all } j \in [M]] > \varepsilon \geq \frac{|D|}{|\mathbb{F}|}$$

Since $\mathbb{x} \notin \mathcal{L}$, some G_{0,j_0} must be δ -far from $\text{RS}^{(0)}$. This is a contradiction to Lemma 5.6 of [BGK⁺23].

- Round 0 to Round 1 : Suppose $(\mathbb{x}, \tau_0) \in \mathcal{D}$, then G_{1,j_0} is δ -far from $\text{RS}^{(1)}$ for some $j_0 \in [M]$. If $\Pr_{\mathbf{c}_1 \leftarrow \mathbb{F}} [(\mathbb{x}, \tau_0 \parallel \mathbf{G}_1^* \parallel \mathbf{c}_1) \notin \mathcal{D}] > \varepsilon$, then

$$\Pr_{\mathbf{c}_1 \leftarrow \mathbb{F}} [\text{CorrectFolding}(1) \wedge G_{2,j} \text{ is } \delta\text{-close to } \text{RS}^{(2)} \text{ for all } j \in [M]] > \varepsilon.$$

This means

$$\Pr_{\mathbf{c}_1 \leftarrow \mathbb{F}} [G_{2,j_0} \text{ is } \delta\text{-close to } \text{RS}^{(2)}] > \varepsilon \geq \frac{|D|}{|\mathbb{F}|}.$$

But since G_{1,j_0} is δ -far from $\text{RS}^{(1)}$, this is a contradiction to Lemma 5.6 of [BGK⁺23].

- Round $i-1$ to Round i , $2 \leq i \leq k-1$: Suppose $(\mathbb{x}, \tau_{i-1}) \in \mathcal{D}$, then we either have $\text{IncorrectFolding}(i-1)$ or G_{i,j_0} is δ -far from $\text{RS}^{(i)}$ for some $j_0 \in [M]$. If $\Pr_{\mathbf{c}_i \leftarrow \mathbb{F}} [(\mathbb{x}, \tau_{i-1} \parallel \mathbf{G}_i^* \parallel \mathbf{c}_i) \notin \mathcal{D}] > \varepsilon$, then

$$\Pr_{\mathbf{c}_i \leftarrow \mathbb{F}} [\text{CorrectFolding}(i) \wedge G_{i+1,j} \text{ is } \delta\text{-close to } \text{RS}^{(i+1)} \text{ for all } j \in [M]] > \varepsilon.$$

This implies $\text{CorrectFolding}(i-1)$, so G_{i,j_0} is δ -far from $\text{RS}^{(i)}$ for some $j_0 \in [M]$. The last probability also implies

$$\Pr_{\mathbf{c}_i \leftarrow \mathbb{F}} [G_{i+1,j_0} \text{ is } \delta\text{-close to } \text{RS}^{(i+1)}] > \varepsilon \geq \frac{|D|}{|\mathbb{F}|},$$

contradicting Lemma 5.6 of [BGK⁺23].

- Round $k-1$ to Round k : Suppose $\Pr_{\mathbf{s} \leftarrow \mathbb{F}} [(\mathbb{x}, \tau_{k-1} \parallel \mathbf{G}_k^* \parallel \mathbf{s}) \notin \mathcal{D}] > \varepsilon$, then

$$\Pr_{\mathbf{s} \leftarrow \mathbb{F}} [\mathcal{V} \text{ accepts}] > \varepsilon \geq (1 - \delta)^l.$$

Since $\mathbb{x} \notin \mathcal{L}$, G_{0,j_0} is δ -far from $\text{RS}^{(0)}$ for some j_0 . However, we always have $G_{k,j}^* \in \mathbb{F}$ which represents a codeword in $\text{RS}^{(k)}$ for all j . This is a contradiction to Lemma 5.4 of [BGK⁺23]. $\square$

4.2.2 Round-by-round Knowledge Soundness of Fold-and-Batch

Before defining a doomed set for Fold-and-Batch, we describe its transcripts:

- **(Round 0 to Round $k - 1$)** For these partial FRI folding rounds, the i -th round partial transcript is $\tau_i := \tau_{i-1} \parallel \mathbf{G}_i^* \parallel \mathbf{c}_i$ where $\mathbf{c}_i = (c_{i,1}, \dots, c_{i,M})$ contains the random folding challenges sent by the verifier and will be used by the provers to obtain the next folded local polynomial.
- **(Round k)** This is the last partial FRI folding round, and $\tau_k := \tau_{k-1} \parallel \mathbf{G}_k^* \parallel \theta$. Instead of FRI folding challenges, the verifier sends the batched FRI challenge θ .
- **(Round $k + 1$ to Round $k + r$)** During these rounds, the (honest) master prover performs the FRI folding of the batched polynomial $F_0 = \sum_{j=1}^M \theta^{j-1} G_{k,j}$. For $i \in \{0, \dots, r-1\}$, the $(k+1+i)$ -th round partial transcript is $\tau_{k+1+i} := \tau_{k+i} \parallel F_i^* \parallel y_i$.
- **(Round $k + r + 1$)** This is the last round of the batched FRI phase and the entire protocol. We have $\tau_{k+r+1} := \tau_{k+r} \parallel F_r^* \parallel \mathbf{s}$ where $\mathbf{s}$ contains the FRI query positions sampled from the evaluation domain D by the verifier.

A complete transcript for $\Pi_{\text{Fold-and-Batch}}$ has the following format:

$$\mathbf{G}_0 \parallel \mathbf{c}_0 \parallel \mathbf{G}_1^* \parallel \mathbf{c}_1 \parallel \dots \parallel \mathbf{c}_{k-1} \parallel \mathbf{G}_k^* \parallel \theta \parallel F_0^* \parallel y_0 \parallel F_1^* \parallel y_1 \parallel \dots \parallel y_{r-1} \parallel F_r^* \parallel \mathbf{s}$$

Definition 9 (Doomed Set for Fold-and-Batch). The doomed set $\mathcal{D}$ for Fold-and-Batch is defined as follows:

1. $(\mathbb{x}, \emptyset) \in \mathcal{D}$ for all possible $\mathbb{x}$.
2. If $\mathbb{x} \notin \mathcal{L}$, then $(\mathbb{x}, \tau_0) \in \mathcal{D}$ if and only if $G_{1,j}$ is δ -far from $\mathbf{RS}^{(1)}$ for some $j \in [M]$.
3. If $\mathbb{x} \notin \mathcal{L}$ and $i \in \{1, \dots, k-1\}$, then $(\mathbb{x}, \tau_i) \in \mathcal{D} \iff \text{IncorrectFolding}(i)$ or $G_{i+1,j}$ is δ -far from $\mathbf{RS}^{(i+1)}$ for some $j \in [M]$.
4. If $\mathbb{x} \notin \mathcal{L}$ and $i \in \{0, \dots, r\}$, then $(\mathbb{x}, \tau_{k+i}) \in \mathcal{D} \iff \text{IncorrectFolding}(k+i)$ or F_i is δ -far from $\mathbf{RS}^{(k+i)}$.
5. If $\mathbb{x} \notin \mathcal{L}$, then $(\mathbb{x}, \tau_{k+r+1}) \in \mathcal{D} \iff$ one of the following consistency checks performed by the verifier fails: (1) consistency between $G_{i,j}^*$ and $G_{i+1,j}^*$ for any $j \in [M]$; (2) consistency between F_0^* and $\{G_{k,j}^*\}_{j=1}^M$; and (3) consistency between F_i^* and F_{i+1}^* .

Proposition 7 (Fold-and-Batch round-by-round knowledge soundness). Define the language $\mathcal{L} := \mathcal{L}_{\mathbf{RS}, \delta}^M$ where $\mathcal{L}_{\mathbf{RS}, \delta} = \{w \in \mathbb{F}^D \mid \Delta(w, \mathbf{RS}) < \delta\}$ is the language of words that are δ -close to $\mathbf{RS}$. Assume the Fold-and-Batch IOP $\Pi_{\text{Fold-and-Batch}}$ is run with $\delta < \frac{1-\rho}{2}$ (unique decoding radius), then it is an IOP for $\mathcal{L}$ with round-by-round knowledge error

$$\varepsilon := \max \left\{ \frac{|D| \cdot (M-1)}{|\mathbb{F}|}, (1-\delta)^l \right\}$$

Proof. Properties (1) and (2) for round-by-round knowledge soundness follow from the definition of the doomed set $\mathcal{D}$. For property (3), we prove it for the cases where $\mathbb{x} \in \mathcal{L}$ and $\mathbb{x} \notin \mathcal{L}$. For all the rounds when $\mathbb{x} \in \mathcal{L}$ and up to round $k-1$ for $\mathbb{x} \notin \mathcal{L}$, the proof is the same as in Proposition 6. We start the analysis from round $k-1$ for $\mathbb{x} \notin \mathcal{L}$:

- Round $k - 1$ to Round k : Suppose $(\mathbb{x}, \tau_{k-1}) \in \mathcal{D}$, then $\text{IncorrectFolding}(k - 1)$ or G_{k,j_0} is δ -far from $\text{RS}^{(k)}$ for some $j_0 \in [M]$. If $\Pr_{\theta \leftarrow \mathbb{F}}[(\mathbb{x}, \tau_{k-1} \parallel \mathbf{G}_k^* \parallel \theta) \notin \mathcal{D}] > \varepsilon$, then

$$\Pr_{\theta \leftarrow \mathbb{F}}[\text{CorrectFolding}(k) \wedge F_0 = \sum_{j=1}^M \theta^{j-1} G_{k,j} \text{ is } \delta\text{-close to } \text{RS}^{(k)}] > \varepsilon.$$

This implies $\text{CorrectFolding}(k)$, so G_{k,j_0} is δ -far from $\text{RS}^{(k)}$ for some $j_0 \in [M]$. However, the last probability implies

$$\Pr_{\theta \leftarrow \mathbb{F}}[F_0 = \sum_{j=1}^M \theta^{j-1} G_{k,j} \text{ is } \delta\text{-close to } \text{RS}^{(k)}] > \varepsilon \geq \frac{|D| \cdot (M - 1)}{|\mathbb{F}|},$$

contradicting Lemma 5.10 of [BGK⁺23].

- Round k to Round $k + 1$: Suppose $(\mathbb{x}, \tau_k) \in \mathcal{D}$, then $\text{IncorrectFolding}(k)$ or F_0 is δ -far from $\text{RS}^{(k)}$. If $\Pr_{y_0 \leftarrow \mathbb{F}}[(\mathbb{x}, \tau_k \parallel F_0^* \parallel y_0) \notin \mathcal{D}] > \varepsilon$, then

$$\Pr_{y_0 \leftarrow \mathbb{F}}[\text{CorrectFolding}(k + 1) \wedge F_1 \text{ is } \delta\text{-close to } \text{RS}^{(k+1)}] > \varepsilon.$$

This implies $\text{CorrectFolding}(k)$, so F_0 is δ -far from $\text{RS}^{(k)}$. However, the last probability implies

$$\Pr_{y_0 \leftarrow \mathbb{F}}[F_1 \text{ is } \delta\text{-close to } \text{RS}^{(k+1)}] > \varepsilon \geq \frac{|D|}{|\mathbb{F}|},$$

contradicting Lemma 5.6 of [BGK⁺23].

- Round $i - 1$ to Round i , $k + 2 \leq i \leq k + r$: We omit this part as it is the same as the analysis for FRI in the proof of Theorem 5.11 in [BGK⁺23].
- Round $k + r$ to Round $k + r + 1$: Suppose $(\mathbb{x}, \tau_{k+r}) \in \mathcal{D}$. Then, we have that $\text{IncorrectFolding}(k + r)$ or F_r is δ -far from $\text{RS}^{(k+r)}$. However, we always have $F_r \in \text{RS}^{(k+r)}$, therefore $\text{IncorrectFolding}(k + r) = \text{True}$. If $\Pr_{\mathbf{s} \leftarrow \mathbb{F}}[(\mathbb{x}, \tau_{k+r} \parallel F_r^* \parallel \mathbf{s}) \notin \mathcal{D}] > \varepsilon$, then

$$\Pr_{\mathbf{s} \leftarrow \mathbb{F}}[\mathcal{V} \text{ accepts the complete transcript } \tau_{k+r+1}] > \varepsilon.$$

Suppose an incorrect folding happened during the partial FRI folding phase, so $\text{IncorrectFolding}(k) = \text{True}$. Let τ_k denote the prefix of τ_{k+r+1} that includes messages up to and including the k -th round, the last round of partial FRI folding. Let τ denote any “honest” extension of τ_k to a complete Parallel FRI transcript. That is, the Parallel FRI prover only behaves honestly after round k . If $\mathcal{V}_{\text{partialFRI}}$ denotes the verifier that checks the partial FRI folding proof, then

$$\Pr_{\mathbf{s} \leftarrow \mathbb{F}}[\mathcal{V}_{\text{partialFRI}} \text{ accepts } \tau_k] = \Pr_{\mathbf{s} \leftarrow \mathbb{F}}[\mathcal{V}_{\text{ParallelFRI}} \text{ accepts } \tau]$$

because any inconsistency detected by $\mathcal{V}_{\text{ParallelFRI}}$ can only be from the first k rounds. Since $\text{IncorrectFolding}(k) = \text{True}$, by Proposition 6 we have

$$\Pr_{\mathbf{s} \leftarrow \mathbb{F}}[\mathcal{V}_{\text{partialFRI}} \text{ accepts } \tau_k] = \Pr_{\mathbf{s} \leftarrow \mathbb{F}}[\mathcal{V}_{\text{ParallelFRI}} \text{ accepts } \tau] \leq \varepsilon_{\text{ParallelFRI}} \leq \varepsilon$$

which is a contradiction. Therefore, the incorrect folding happened during the batched FRI phase. Since $\mathbf{x} \notin \mathcal{L}$, some G_{0,j_0} is δ -far from $\mathbf{RS}^{(0)}$. By Lemma 5.4 of [BGK⁺23], G_{k,j_0}^* must be δ -far from $\mathbf{RS}^{(k)}$. Therefore, the input $\mathbf{x}_{\text{bFRI}}$ to the batched FRI phase is not in $\mathcal{L}_{\text{bFRI}}$. Let $\mathcal{V}_{\text{bFRI}}$ denote the verifier that checks the batched FRI proof. Since $\Pr_{\mathbf{s} \xleftarrow{\$} \mathbb{F}}[\mathcal{V}_{\text{bFRI}} \text{ accepts}] > \varepsilon$, all the foldings performed up to and including the $(k+r)$ -th round during the batched FRI phase must be honest by the round-by-round knowledge soundness of batched FRI. This is again a contradiction. $\square$

4.2.3 Concrete FRI Parameters

In this section, we explain how to compute concrete FRI parameters for our PCS. We will closely follow the notations and method used in Appendix A of [BGK⁺23]. We will focus solely on the Fold-and-Batch variant. The computations for Parallel FRI and Distributed Batched FRI are similar.

In Proposition 7, we proved an explicit round-by-round knowledge soundness bound for the Fold-and-Batch IOP. This enables one to compute concrete FRI parameters to set for a target security level. In Table 3, we provide three sets of such parameters, targeting the 80-bit, 100-bit, and 128-bit security levels.

Let Π be an IOP and let $\mathbf{BCS}[\Pi]$ be the corresponding BCS-transformed non-interactive protocol. Let ε_{rbr} denote the round-by-round knowledge soundness error of Π . Let ε_{FS} denote the knowledge soundness error of $\mathbf{BCS}[\Pi]$. Let κ denote the number of bits in the output of the random oracle. Let Q be the number of random oracle queries that an IOP attacker can make, which is roughly the number of hashes an attacker can perform.

By Theorem 3.15 of [BGK⁺23], we have the following relation:

$$\varepsilon_{\text{FS}} = Q \cdot \varepsilon_{\text{rbr}} + \frac{3(Q^2 + 1)}{2^\kappa}$$

In order for ε_{FS} to be upper bounded by $2^{-\nu}$ where ν is the desired bits of security, we first bound both terms by $\frac{2^{-\nu}}{2}$. We set $\kappa = 256$, so

$$\frac{3(Q^2 + 1)}{2^\kappa} \leq \frac{2^{-\nu}}{2}$$

gives us an upper bound for Q . Then, setting Q to be the maximum possible value (i.e., assuming the attacker has the maximum amount of computational power within the limit we consider), the inequality

$$Q \cdot \varepsilon_{\text{rbr}} \leq \frac{2^{-\nu}}{2}$$

gives us an upper bound S for ε_{rbr} . Using the explicit bound

$$\varepsilon_{\text{rbr}} = \max \left\{ \frac{|D| \cdot (M-1)}{|\mathbb{F}|}, (1-\delta)^l \right\}$$

proven in Proposition 7, we can now obtain the FRI parameters by setting $\frac{|D| \cdot (M-1)}{|\mathbb{F}|} \leq S$ and $(1-\delta)^l < S$. Recall that we assume the Reed-Solomon proximity parameter δ is within the unique decoding radius $\frac{1-\rho}{2}$. In Table 3, for each choice of code rate ρ , we always choose the maximum possible δ to minimize the number of FRI queries l .

4.2.4 Non-interactive Fold-and-Batch

To turn the (interactive) Fold-and-Batch described in Section 4 into a non-interactive protocol, we use the Fiat-Shamir transformation [FS86]. In Figures 2 and 3 below, we

Table 3: Fold-and-Batch parameter recommendations for various security levels. ν is the number of bits of security. Q is the allowed number of random oracle queries made by a malicious prover. ε_{rbr} is the round-by-round knowledge soundness of DFRI. ρ is the code rate. l is the smallest number of FRI queries to achieve the desired security level for a given FRI blowup factor. $|\mathbb{F}|$ is the size of the field. $|D|$ is the size of the FRI evaluation domain for worker nodes. M is the number of workers.

ν	Q	ε_{rbr}	$\rho = \frac{1}{2}$	$\rho = \frac{1}{4}$	$\rho = \frac{1}{8}$	$\rho = \frac{1}{16}$	$ \mathbb{F} $
128	$\leq 2^{62}$	$< 2^{-191}$	$l = 461$	$l = 282$	$l = 231$	$l = 210$	$\geq 2^{191} \cdot D (M-1)$
100	$\leq 2^{76}$	$< 2^{-177}$	$l = 427$	$l = 262$	$l = 214$	$l = 194$	$\geq 2^{177} \cdot D (M-1)$
80	$\leq 2^{86}$	$< 2^{-167}$	$l = 403$	$l = 247$	$l = 202$	$l = 184$	$\geq 2^{167} \cdot D (M-1)$

illustrate how the prover derives the Fiat-Shamir challenges on a small toy example by using the hash chain variant of Fiat-Shamir described in Section 2.1 of [CO25]. The security of this variant is proven in Chapter 15 of [CY24]. In this variant, the prover has access to $k+1$ random oracles in a k -round protocol, one for the Merkle commitment scheme and one for deriving the verifier’s random challenge in each round. We adapt this setup to the distributed setting, by using M independently sampled random oracles for each round of the folding phase.

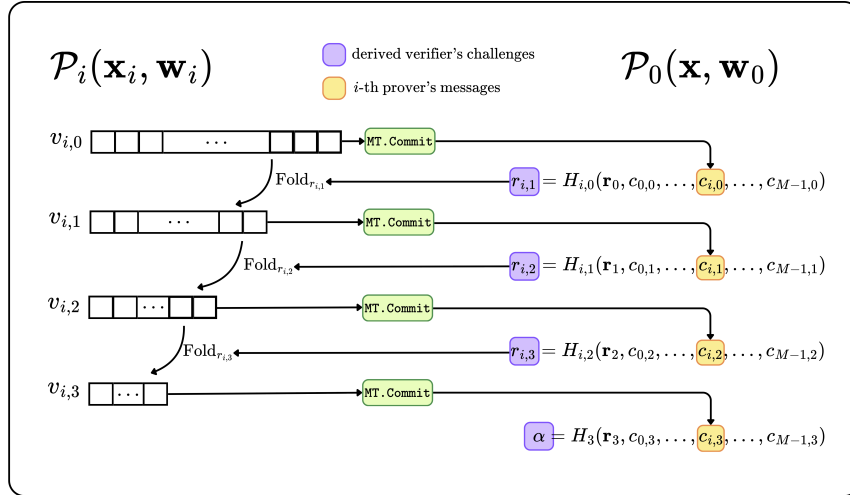


Figure 2: The derivation of Fiat-Shamir challenges for Fold-and-Batch folding phase, illustrated using a toy example (partly inspired by Figure 15.1 in [CY24]).

Domain Separation. For domain separation, we adopt the common technique to instantiate multiple random oracles using a single hash function, by prepending the random oracle inputs with a unique binary string associated with that domain. Let H be a cryptographic hash function. We instantiate the random oracles as follows: (1) The random oracle used by the Merkle commitment scheme is instantiated using $H_{\text{MT}}(x) := H(0 \parallel x)$, (2) the random oracle used by the i -th prover node in the j -round is instantiated with $H_{i,j}(x) := H(1 \parallel i \parallel j \parallel x)$, where $\parallel$ denotes concatenation of binary strings and each one of the integers $0, 1, i, j$ is represented using a unique binary string of fixed length s .

Deriving the FS Challenges. To derive the next FRI folding challenge (i.e., $r_{i,j}$ ’s and r_j ’s), each worker node computes the Merkle commitment of its local polynomial in the

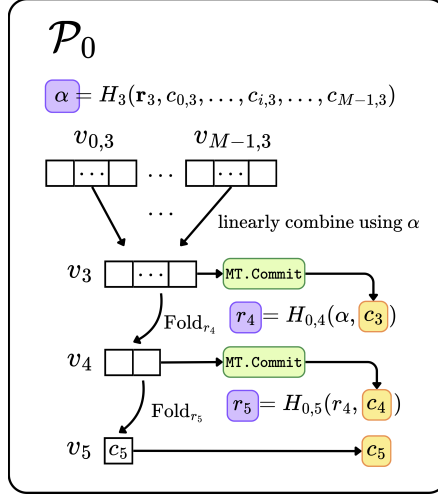


Figure 3: The derivation of Fiat-Shamir challenges for Fold-and-Batch batched FRI phase, illustrated using a toy example (partly inspired by Figure 15.1 in [CY24]).

current round and sends the Merkle root to the master prover. The master prover then derives the next FRI folding challenge for the i -th worker node by hashing the verifier’s challenges for all nodes from the previous round and the Merkle roots sent by all nodes in the current round. This hash is computed using a hash function dedicated to the current round and the i -th worker node. For example, in the j -th round, the FRI folding challenge for the i -th worker node is computed using the hash function $H_{i,j}$. In Figures 2 and 3, we use $\mathbf{r}_j := (r_{0,j}, \dots, r_{M-1,j})$ to denote the M verifier’s challenges from round j . For round 0, if the $c_{i,0}$ ’s are part of the first prover message in the protocol, then the master node derives the next verifier’s challenges by setting $\mathbf{r}_0 := \mathbf{x}$, otherwise it sets $\mathbf{r}_0$ to be the M verifier’s challenges derived in the previous round. During the batched FRI phase, the master node derives the FRI folding challenge similarly by hashing the verifier’s challenge from the previous round and the Merkle root it computes in the current round without communicating with the worker nodes.

5 Implementation and Evaluation

In this section, we present the implementation details of the three DFRI methods and compare their performance. We implemented all three variants of DFRI using 3700+ lines of code in Rust. Our implementation² is based on the STARK implementation winterfell. The experiments were run on a Rocky Linux 9 server with 48 cores and 377 GiB of RAM.

Choice of parameters. For all experiments, we used a quadratic extension $\mathbb{F} := \mathbb{F}_{p^2}$ of a prime field $\mathbb{F}_p$ with modulus $p = 2^{128} - 45 \cdot 2^{40} + 1$. This field satisfies $2^{41} \mid |\mathbb{F}^*|$, hence the size of the FRI evaluation domain $|D| = T\rho^{-1}$ must be no larger than 2^{41} . We set the FRI blowup factor to $\rho^{-1} = 4$ and the number of FRI queries to $l = 282$. According to Table 3, this provides 128 bits of security for the non-interactive Fold-and-Batch. In fact, these parameters provide the same security level for Parallel FRI and Distributed Batched FRI, as the Fold-and-Batch round-by-round knowledge error upper bounds those of the other two variants.

We conducted the experiments on circuit sizes $N = 2^{15}$, 2^{20} and 2^{25} . We benchmarked

²<https://github.com/XuHuaXH/winterfell>

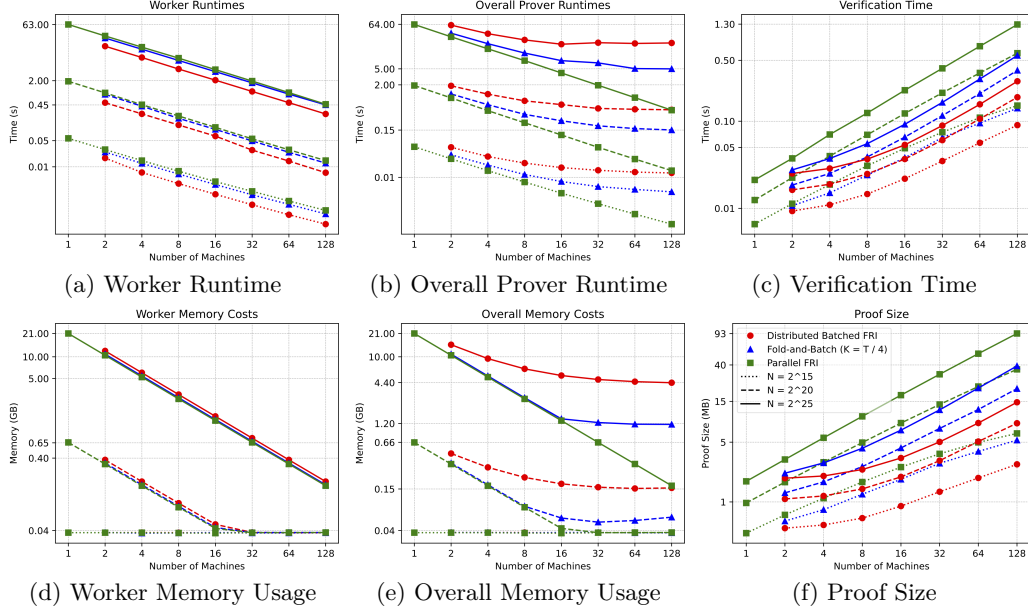


Figure 4: Performance graphs.

all DFRI methods on up to 128 machines. For Fold-and-Batch, we use a folding parameter $K = \frac{T}{4}$, i.e., each worker node folds its local polynomial twice using a folding factor of 2. The Merkle trees used in FRI are instantiated with the BLAKE3³ hash function with 256-bit outputs. Winterfell provides a BLAKE3 implementation with 192-bit and 256-bit outputs. However, the 192-bit output option only provides at most 96 bits of security⁴. Instead of setting up multiple machines, we simulated all the machines on a single one by running them sequentially. Therefore, the measured prover runtime does not include the time spent on network communications.

We present the measurements on several performance metrics and discuss how the three DFRI methods compare on each metric.

Worker node. In Figures 4a and 4d, we show the cost of a single worker node during proof generation. All three DFRI methods are perfectly scalable for worker nodes. For circuit size $N = 2^{25}$, increasing the number of machines from 2 to 128 results in at least $61\times$ faster proving time; the memory usage decreases by at least $63\times$ for all three methods.

Overall prover cost. Figures 4b and 4e show the cost for provers overall. The *overall memory* is the peak memory usage measured at the master node during proof generation. Since the master node also acts as one of the worker nodes, this number represents the maximum memory requirement for a prover node. The data points corresponding to 1 machine show the costs for monolithic FRI. For parallel FRI, there is no master node, and each prover node does the same amount of work. Hence, the overall prover cost is the same as that of a single worker node. Fold-and-Batch is somewhat scalable, and Distributed Batched FRI has only limited scalability. For circuit size 2^{25} , as we increase the number of machines from 1 to 128, the runtime is $2.8\times$ faster for Distributed Batched FRI and $12.6\times$ faster for Fold-and-Batch; the memory usage decreased by $4.7\times$ for Distributed Batched FRI and $17.9\times$ for Fold-and-Batch.

³<https://github.com/BLAKE3-team/BLAKE3-specs>

⁴winterfell documentation: <https://github.com/facebook/winterfell/tree/main/crypto>

Communication cost. The total communication cost for circuit size 2^{25} is around 4.3 GB for Distributed Batched FRI and around 1.1 GB for Fold-and-Batch. For both protocols, these costs come from all the worker nodes sending their evaluation vectors to the master node right before running batched FRI. However, the cost of Fold-and-Batch is approximately four times smaller than that of Distributed Batched FRI. This is because, by setting $K := \frac{T}{4}$, the witness data sent in Fold-and-Batch is four times smaller. Assuming a network bandwidth of 25 Gbps, the communication at circuit size $N = 2^{25}$ adds 1.38s and 0.35s to the overall proving time for Distributed Batched FRI and Fold-and-Batch, respectively.

Verification cost. The verification cost is shown in Figure 4c. For all three DFRI methods, the verification time and proof size increase with the number of machines. For circuit size 2^{25} , as we increase from 1 node to 128 nodes, the verification time is $13.6\times$ slower for Distributed Batched FRI, $26.8\times$ slower for Fold-and-Batch, and $60.8\times$ slower for Parallel FRI. The proof size is $8.4\times$ larger for Distributed Batched FRI, $22.4\times$ larger for Fold-and-Batch, and $53.6\times$ larger for Parallel FRI.

Among the three DFRI methods, Parallel FRI has the smallest and perfectly scalable prover cost while having the most expensive verifier cost. Distributed Batched FRI has the most expensive and least scalable prover while having the smallest verifier cost. For Fold-and-Batch with a folding parameter $K = \frac{T}{4}$, its prover and verifier costs are both between the two other methods. In conclusion, there is a prover-verifier trade-off for DFRI, with Parallel FRI being the most prover-friendly, Distributed Batched FRI being the most verifier-friendly, and Fold-and-Batch in between. We summarize this performance trade-off in Table 4.

Table 4: DFRI Variants Comparison Table

DFRI variant	Prover performance and scalability	Comm.	Verification
Parallel FRI	Efficient, perfectly scalable	No cost	Expensive
Distributed Batched FRI	Slow, limited scalability	Expensive	Cheap
Fold-and-Batch ($K = \frac{T}{4}$)	Moderate performance and scalability	Moderate	Moderate

Barrier preventing a verification cost independent of M . The verification cost in our FRI-based setting contrasts with *Pianist*, which has an $O(1)$ verification cost. The fact that *Pianist*’s verification cost is independent of the number of machines allows it to scale linearly without negatively affecting the verifier. The main barrier to achieving such a performance profile in the FRI-based setting is the lack of homomorphism in the Merkle commitment scheme. Once the commitments are computed, the prover cannot efficiently compute an “aggregation” of these commitments without exchanging witness data. For *Pianist*, it uses the DKZG (Distributed KZG) polynomial commitment scheme, which commits by exponentiating group elements. Since group exponentiation is homomorphic, the master node can combine any number of commitments/partial proofs sent from the worker nodes into a single group element.

Acknowledgments

This work was supported by the Flemish Government through the Cybersecurity Research Program with grant number VOEWICS02. We thank Jannik Spiessens for valuable discussions on the BCS transform and the anonymous CiC reviewers for their helpful feedback on this paper.

References

- [0xP21] 0xPolygonZero. Plonky2, 2021. URL: <https://github.com/0xPolygonZero/plonky2/blob/main/plonky2/plonky2.pdf>.
- [BBHR18] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast reed-solomon interactive oracle proofs of proximity. In Ioannis Chatzigiannakis, Christos Kaklamanis, Dániel Marx, and Donald Sannella, editors, *45th International Colloquium on Automata, Languages, and Programming, ICALP 2018, Prague, Czech Republic, July 9-13, 2018*, volume 107 of *LIPICs*, pages 14:1–14:17. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2018. URL: <https://doi.org/10.4230/LIPICs.ICALP.2018.14>, doi:10.4230/LIPICs.ICALP.2018.14.
- [BCI⁺20] Eli Ben-Sasson, Dan Carmon, Yuval Ishai, Swastik Kopparty, and Shubhangi Saraf. Proximity gaps for reed-solomon codes. In *61st Annual Symposium on Foundations of Computer Science*, pages 900–909, Durham, NC, USA, November 16–19, 2020. IEEE Computer Society Press. doi:10.1109/FOCS46700.2020.00088.
- [BFS20] Benedikt Bünz, Ben Fisch, and Alan Szepieniec. Transparent snarks from DARK compilers. In Anne Canteaut and Yuval Ishai, editors, *Advances in Cryptology - EUROCRYPT 2020 - 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10-14, 2020, Proceedings, Part I*, volume 12105 of *Lecture Notes in Computer Science*, pages 677–706. Springer, 2020. doi:10.1007/978-3-030-45721-1_24.
- [BGK⁺23] Alexander R. Block, Albert Garreta, Jonathan Katz, Justin Thaler, Pratyush Ranjan Tiwari, and Michal Zajac. Fiat-shamir security of FRI and related snarks. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology - ASIACRYPT 2023 - 29th International Conference on the Theory and Application of Cryptology and Information Security, Guangzhou, China, December 4-8, 2023, Proceedings, Part II*, volume 14439 of *Lecture Notes in Computer Science*, pages 3–40. Springer, 2023. doi:10.1007/978-981-99-8724-5_1.
- [BSCS16] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. Interactive oracle proofs. In *Proceedings, Part II, of the 14th International Conference on Theory of Cryptography - Volume 9986*, page 31–60, Berlin, Heidelberg, 2016. Springer-Verlag. doi:10.1007/978-3-662-53644-5_2.
- [CBBZ23] Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. Hyperplonk: Plonk with linear-time prover and high-degree custom gates. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23-27, 2023, Proceedings, Part II*, volume 14005 of *Lecture Notes in Computer Science*, pages 499–530. Springer, 2023. doi:10.1007/978-3-031-30617-4_17.
- [Che24] Alisa Cherniaeva. On distributed fri-based proof generation - hackmd, 10 2024. URL: https://hackmd.io/@nil-research/rJ_NVyiRA.
- [CMS19] Alessandro Chiesa, Peter Manohar, and Nicholas Spooner. Succinct arguments in the quantum random oracle model. In *Theory of Cryptography: 17th International Conference, TCC 2019, Nuremberg, Germany, December 1–5*,

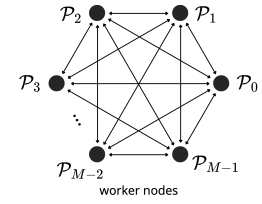
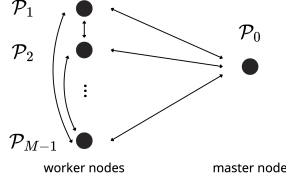
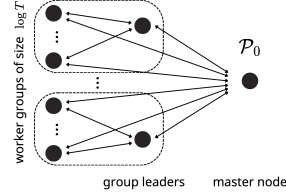
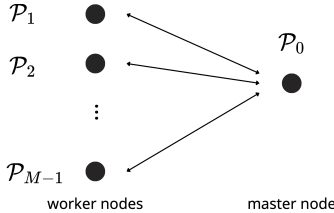
- 2019, *Proceedings, Part II*, page 1–29, Berlin, Heidelberg, 2019. Springer-Verlag. doi:[10.1007/978-3-030-36033-7_1](https://doi.org/10.1007/978-3-030-36033-7_1).
- [CO25] Alessandro Chiesa and Michele Orrù. A fiat-shamir transformation from duplex sponges. In *Theory of Cryptography: 23rd International Conference, TCC 2025, Aarhus, Denmark, December 1–5, 2025, Proceedings, Part I*, page 452–474, Berlin, Heidelberg, 2025. Springer-Verlag. doi:[10.1007/978-3-032-12287-2_16](https://doi.org/10.1007/978-3-032-12287-2_16).
- [CY24] Alessandro Chiesa and Eylon Yogev. *Building Cryptographic Proofs from Hash Functions*. 09 2024. URL: <https://hash-based-snargs-book.github.io/>.
- [FS86] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In Andrew M. Odlyzko, editor, *Advances in Cryptology - CRYPTO '86, Santa Barbara, California, USA, 1986, Proceedings*, volume 263 of *Lecture Notes in Computer Science*, pages 186–194. Springer, 1986. doi:[10.1007/3-540-47721-7_12](https://doi.org/10.1007/3-540-47721-7_12).
- [Gro16] Jens Groth. On the size of pairing-based non-interactive arguments. In Marc Fischlin and Jean-Sébastien Coron, editors, *Advances in Cryptology – EUROCRYPT 2016, Part II*, volume 9666 of *Lecture Notes in Computer Science*, pages 305–326, Vienna, Austria, May 8–12, 2016. Springer Berlin Heidelberg, Germany. doi:[10.1007/978-3-662-49896-5_11](https://doi.org/10.1007/978-3-662-49896-5_11).
- [GWC19] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. PLONK: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. *Cryptology ePrint Archive*, Paper 2019/953, 2019. URL: <https://eprint.iacr.org/2019/953>.
- [KPPS20] Ahmed E. Kosba, Dimitrios Papadopoulos, Charalampos Papamanthou, and Dawn Song. MIRAGE: succinct arguments for randomized algorithms with applications to universal zk-snarks. In Srdjan Capkun and Franziska Roesner, editors, *29th USENIX Security Symposium, USENIX Security 2020, August 12–14, 2020*, pages 2129–2146. USENIX Association, 2020. URL: <https://www.usenix.org/conference/usenixsecurity20/presentation/kosba>.
- [KZG10] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In Masayuki Abe, editor, *Advances in Cryptology – ASIACRYPT 2010*, volume 6477 of *Lecture Notes in Computer Science*, pages 177–194, Singapore, December 5–9, 2010. Springer Berlin Heidelberg, Germany. doi:[10.1007/978-3-642-17373-8_11](https://doi.org/10.1007/978-3-642-17373-8_11).
- [Lee21] Jonathan Lee. Dory: Efficient, transparent arguments for generalised inner products and polynomial commitments. In *Theory of Cryptography: 19th International Conference, TCC 2021, Raleigh, NC, USA, November 8–11, 2021, Proceedings, Part II*, page 1–34, Berlin, Heidelberg, 2021. Springer-Verlag. doi:[10.1007/978-3-030-90453-1_1](https://doi.org/10.1007/978-3-030-90453-1_1).
- [Lin01] Yehuda Lindell. Parallel coin-tossing and constant-round secure two-party computation. In Joe Kilian, editor, *Advances in Cryptology - CRYPTO 2001, 21st Annual International Cryptology Conference, Santa Barbara, California, USA, August 19–23, 2001, Proceedings*, volume 2139 of *Lecture Notes in Computer Science*, pages 171–189. Springer, 2001. doi:[10.1007/3-540-44647-8_10](https://doi.org/10.1007/3-540-44647-8_10).
- [LXZ⁺24] Tianyi Liu, Tiancheng Xie, Jiaheng Zhang, Dawn Song, and Yupeng Zhang. Pianist: Scalable zkRollups via fully distributed zero-knowledge proofs. In *2024*

- IEEE Symposium on Security and Privacy*, pages 1777–1793, San Francisco, CA, USA, May 19–23, 2024. IEEE Computer Society Press. doi:10.1109/SP54263.2024.00035.
- [LZL⁺25a] Chongrong Li, Pengfei Zhu, Yun Li, Cheng Hong, Wenjie Qu, and Jiaheng Zhang. HyperPianist: Pianist with Linear-Time Prover and Logarithmic Communication Cost. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 3383–3401, Los Alamitos, CA, USA, May 2025. IEEE Computer Society. URL: <https://doi.ieeecomputersociety.org/10.1109/SP61157.2025.00202>, doi:10.1109/SP61157.2025.00202.
- [LZL⁺25b] Weihai Li, Zongyang Zhang, Yun Li, Pengfei Zhu, Cheng Hong, and Jianwei Liu. Soloist: Distributed SNARKs for rank-one constraint system. *Cryptology ePrint Archive*, Paper 2025/557, 2025. URL: <https://eprint.iacr.org/2025/557>.
- [PV07] Rafael Pass and Muthuramakrishnan Venkitasubramaniam. An efficient parallel repetition theorem for arthur-merlin games. In *Proceedings of the Thirty-Ninth Annual ACM Symposium on Theory of Computing*, STOC '07, page 420–429, New York, NY, USA, 2007. Association for Computing Machinery. doi:10.1145/1250790.1250853.
- [RMH⁺24] Michael Rosenberg, Tushar Mopuri, Hossein Hafezi, Ian Miers, and Pratyush Mishra. Hekaton: Horizontally-scalable zkSNARKs via proof aggregation. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*, CCS '24, page 929–940, New York, NY, USA, 2024. Association for Computing Machinery. doi:10.1145/3658644.3690282.
- [VP19] Alexander Vlasov and Konstantin Panarin. Transparent polynomial commitment scheme with polylogarithmic communication complexity. *Cryptology ePrint Archive*, Paper 2019/1020, 2019. URL: <https://eprint.iacr.org/2019/1020>.
- [WSVZ24] Wenhao Wang, Fangyan Shi, Dani Vilardell, and Fan Zhang. Cirrus: Performant and accountable distributed SNARK. *Cryptology ePrint Archive*, Paper 2024/1873, 2024. URL: <https://eprint.iacr.org/2024/1873>.
- [WZC⁺18] Howard Wu, Wenting Zheng, Alessandro Chiesa, Raluca Ada Popa, and Ion Stoica. DIZK: A distributed zero knowledge proof system. In *27th USENIX Security Symposium (USENIX Security 18)*, pages 675–692, Baltimore, MD, August 2018. USENIX Association. URL: <https://www.usenix.org/conference/usenixsecurity18/presentation/wu>.
- [XZC⁺22] Tiancheng Xie, Jiaheng Zhang, Zerui Cheng, Fan Zhang, Yupeng Zhang, Yongzheng Jia, Dan Boneh, and Dawn Song. zkbridge: Trustless cross-chain bridges made practical. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, CCS '22, page 3003–3017, New York, NY, USA, 2022. Association for Computing Machinery. doi:10.1145/3548606.3560652.
- [ZXZS20] Jiaheng Zhang, Tiancheng Xie, Yupeng Zhang, and Dawn Song. Transparent polynomial delegation and its applications to zero knowledge proof. In *2020 IEEE Symposium on Security and Privacy (SP)*, pages 859–876, 2020. doi:10.1109/SP40000.2020.00052.

A Communication patterns and security assumptions

In this section, we complement the related work review from Section 1.1 with further details regarding communication patterns and security assumptions, summarized in Table 5.

Table 5: Security assumptions and communication patterns among prover nodes of distributed proof systems. “Our PCS” refers to our bivariate PCS instantiated with Fold-and-Batch.

System	Security assumptions	Communication pattern
DIZK	q -SBDH	
deVirgo	CRHF	
Cirrus	q -SBDH	
Pianist	q -SBDH	
Hekaton	q -SBDH	
HyperPianist ^K	q -SBDH	
HyperPianist ^D	SXDH	
Soloist	q -SBDH	
Our PCS + Pianist PIOP	CRHF	

Pianist, Hekaton, HyperPianist, and Soloist share a similar communication pattern, where worker nodes communicate only with a master node; the proof scheme obtained from instantiating the Pianist PIOP with Fold-and-Batch also follows this pattern. DeVirgo and Cirrus also utilize a master node to coordinate computations and generate the final proof; however, in these two proof schemes, worker nodes also communicate with one another. In deVirgo, all worker nodes must exchange information with each other at one point in the protocol (see [XZC⁺22, Protocol 6, Step 2]). In Cirrus, the workload of the master node is reduced by introducing a hierarchical proof aggregation method; some worker nodes are designated as “group leaders” and are responsible for aggregating the proofs from workers within their group. To enable accountability, each worker node must communicate with both its group leader and the master node. DIZK has a distinct communication pattern and does not rely on a central coordinator. The workload is, however, not always symmetrical, as the exact work distribution across the nodes is optimized for each task

(e.g., polynomial arithmetic and multi-scalar multiplications).

Regarding the security assumptions, we highlight that only the collision-resistant hash function (CRHF) assumption is plausibly post-quantum. For most other schemes, their soundness relies on the q -Strong Bilinear Diffie-Hellman (q -SBDH) assumption, which requires the discrete logarithm assumption to hold. The HyperPianist scheme instantiated with the distributed Dory PCS relies on the Symmetric eXternal Diffie-Hellman (SXDH) assumption, which also implies the hardness of the discrete logarithm.

B Background on FRI

Let $f(X) = f_0 + f_1X + \dots + f_{T-1}X^{T-1} \in \mathbb{F}[X]$. The FRI folding phase consists of the following steps:

1. $\mathcal{P}$ sends $\mathcal{V}$ the commitment of f , $\text{MT.Commit}(f)$.
2. $\mathcal{V}$ sends $\mathcal{P}$ a random folding factor $r_1 \xleftarrow{\$} \mathbb{F}$.
3. $\mathcal{P}$ splits $f = f^{(0)}$ into an even part and an odd part by writing it as

$$\begin{aligned} f^{(0)}(X) &= \underbrace{(f_0 + f_2X^2 + \dots + f_{T-2}X^{T-2})}_{\text{even terms}} + \underbrace{(f_1X + f_3X^3 + \dots + f_{T-1}X^{T-1})}_{\text{odd terms}} \\ &= f_{\text{even}}^{(0)}(X^2) + X \cdot f_{\text{odd}}^{(0)}(X^2) \end{aligned}$$

so

$$\begin{aligned} f_{\text{even}}^{(0)}(X) &= f_0 + f_2X + f_4X^2 + \dots + f_{T-2}X^{\frac{T-2}{2}}, \\ f_{\text{odd}}^{(0)}(X) &= f_1 + f_3X + f_5X^2 + \dots + f_{T-1}X^{\frac{T-2}{2}} \end{aligned}$$

$\mathcal{P}$ combines $f_{\text{even}}^{(0)}(X)$ and $f_{\text{odd}}^{(0)}(X)$ using r_1 and obtains

$$f^{(1)}(X) = f_{\text{even}}^{(0)}(X) + r_1 \cdot f_{\text{odd}}^{(0)}(X)$$

$\mathcal{P}$ builds a Merkle tree on the evaluation vector $(f^{(1)}(x))_{x \in D^{(1)}}$ where $D^{(1)} := (D^{(0)})^2 = \{x^2 \mid x \in D^{(0)}\} = \mu_{\frac{n}{2}}$ and sends the Merkle root to $\mathcal{V}$. The new polynomial $f^{(1)}(X)$ has degree about half of $\deg f^{(0)}$ and $D^{(1)}$ is half the size of $D = D^{(0)}$.

4. $\mathcal{V}$ sends $\mathcal{P}$ a random folding factor $r_2 \xleftarrow{\$} \mathbb{F}$.
5. $\mathcal{P}$ repeats the same process as in Step 3: $\mathcal{P}$ write $f^{(1)}$ as

$$f^{(1)}(X) = f_{\text{even}}^{(1)}(X^2) + X \cdot f_{\text{odd}}^{(1)}(X^2)$$

and obtains

$$f^{(2)}(X) = f_{\text{even}}^{(1)}(X) + r_2 \cdot f_{\text{odd}}^{(1)}(X)$$

$\mathcal{P}$ sends $\mathcal{V}$ the Merkle commitment of $f^{(2)}$ evaluated on $D^{(2)} := (D^{(1)})^2 = \mu_{\frac{n}{4}}$.

6. ...

This process repeats for a total of $\log T$ rounds. In the last round, f has been folded to a constant polynomial $f^{(\log T)}(X) = c$, so $\mathcal{P}$ sends $\mathcal{V}$ the constant c instead of a Merkle commitment. This concludes the folding phase.

Alternatively, one can view the folding phase as folding the evaluation vector $(f(x))_{x \in D}$. Recall that in the i -th round of folding, $\mathcal{P}$ writes $f^{(i)}(X)$ as

$$f^{(i)}(X) = f_{\text{even}}^{(i)}(X^2) + X \cdot f_{\text{odd}}^{(i)}(X^2)$$

Thus,

$$f^{(i)}(-X) = f_{\text{even}}^{(i)}(X^2) - X \cdot f_{\text{odd}}^{(i)}(X^2)$$

Combining these two equations, we get

$$\begin{aligned} f_{\text{even}}^{(i)}(X^2) &= \frac{f^{(i)}(X) + f^{(i)}(-X)}{2} \\ f_{\text{odd}}^{(i)}(X^2) &= \frac{f^{(i)}(X) - f^{(i)}(-X)}{2X} \end{aligned}$$

Therefore, $f^{(i+1)}(X^2)$ can be expressed as

$$\begin{aligned} f^{(i+1)}(X^2) &\stackrel{\text{def}}{=} f_{\text{even}}^{(i)}(X^2) + r_{i+1} \cdot f_{\text{odd}}^{(i)}(X^2) \\ &= \frac{f^{(i)}(X) + f^{(i)}(-X)}{2} + r_{i+1} \cdot \frac{f^{(i)}(X) - f^{(i)}(-X)}{2X} \\ &= \frac{X + r_{i+1}}{2X} f^{(i)}(X) + \frac{X - r_{i+1}}{2X} f^{(i)}(-X) \end{aligned}$$

This means, if $\mathcal{P}$ already has the evaluation vector $(f^{(i)}(x))_{x \in D^{(i)}}$ for the i -th round, $\mathcal{P}$ can compute the evaluation vector $(f^{(i+1)}(y))_{y \in D^{(i+1)}}$ for the $(i+1)$ -th round as follows: For any $y \in D^{(i+1)} = (D^{(i)})^2$, find x such that $x^2 = y$. Then $x \in D^{(i)}$ and $f^{(i+1)}(y)$ is a linear combination of $f^{(i)}(x)$ and $f^{(i)}(-x)$:

$$f^{(i+1)}(y) = \frac{x + r_{i+1}}{2x} f^{(i)}(x) + \frac{x - r_{i+1}}{2x} f^{(i)}(-x) \quad (2)$$

After the folding phase, $\mathcal{P}$ and $\mathcal{V}$ run the query phase of FRI. The purpose of the query phase is for $\mathcal{V}$ to check whether $\mathcal{P}$ has folded f correctly by “quizzing” $\mathcal{P}$ at various points $g \in D$. The query phase proceeds as follows: For each $g \xleftarrow{\$} D$ that $\mathcal{V}$ queries,

1. $\mathcal{V}$ sends $\mathcal{P}$ the query point g .
2. $\mathcal{P}$ sends $\mathcal{V}$ the evaluations

$$\begin{aligned} &f^{(0)}(g), f^{(0)}(-g), \\ &f^{(1)}(g^2), f^{(1)}(-g^2), \\ &f^{(2)}(g^4), f^{(2)}(-g^4), \\ &\dots \\ &f^{(\log T)}(g^T), f^{(\log T)}(-g^T) \end{aligned}$$

together with their Merkle authentication paths.

3. $\mathcal{V}$ checks that

- (a) All the Merkle authentication paths are indeed valid opening proofs for their corresponding evaluations.

- (b) For each $1 \leq i \leq \log T$, $f^{(i)}(g^{2^i})$ is indeed computed correctly using Formula 2, i.e. that

$$\begin{aligned} f^{(1)}(g^2) &= \frac{g+r_1}{2g} f^{(0)}(g) + \frac{g-r_1}{2g} f^{(0)}(-g) \\ f^{(2)}(g^4) &= \frac{g^2+r_2}{2g^2} f^{(1)}(g^2) + \frac{g^2-r_2}{2g^2} f^{(1)}(-g^2) \\ &\dots \end{aligned}$$

If both conditions are satisfied, $\mathcal{V}$ outputs **accept**. Otherwise, $\mathcal{V}$ outputs **reject**.

Soundness of FRI One can estimate the number of queries needed in the FRI query phase in order to guarantee λ bits of security using Theorem 3.3 from [BBHR18]: If the function $f : D \rightarrow \mathbb{F}$ is δ -far from **RS**, i.e.

$$\frac{|\{x \in D \mid f(x) \neq F(x)\}|}{|D|} \geq \delta$$

for any $F \in \mathbf{RS}$ and $\mathcal{V}$ sent l queries during the FRI query phase, then

$$\Pr[\mathcal{V} \text{ outputs } \mathbf{accept}] \leq \frac{3|D|}{|\mathbb{F}|} + (1 - \delta)^l$$

Since the term $\frac{3|D|}{|\mathbb{F}|}$ is negligibly small, we get about λ bits of security once we have

$$(1 - \delta)^l < 2^{-\lambda}$$

This means the number of queries l should be at least $-\frac{\lambda}{\log_2(1-\delta)}$. For a fixed security parameter λ , choosing a larger δ allows us to run fewer query rounds, which in turn reduces the proof size and verification time. However, a larger δ means a larger FRI blowup factor ρ^{-1} (since $\delta < \frac{1-\rho}{2}$), which implies longer proving time because the prover needs to fold a larger polynomial during the FRI folding phase.

C Cost analysis of DFRI instantiations

In Tables 6 and 7 we present a step-by-step cost analysis of Distributed Batched FRI and of Fold-and-Batch, respectively.

D Parallel Execution of IOPs

We formally define the parallel execution of multiple IOPs. Then, Parallel FRI can be viewed as the parallel execution of M different instances of FRI, with input $\mathbf{x} = (\mathbf{x}_1, \dots, \mathbf{x}_M)$ where each $\mathbf{x}_j$ is an input for FRI. Note that this is different from parallel repetition (e.g., as in [PV07]), where the inputs to all the parallel instances are the same. We use $[M]$ to denote the set $\{1, \dots, M\}$.

Definition 10 (Parallel Execution of IOPs). For each $j \in [M]$, let $\Pi_j = (\mathcal{P}_j, \mathcal{V}_j)$ be a t -round IOP for a relation $\mathcal{R}_j$ with δ_j -completeness and ε_j -soundness. The *parallel execution of Π_j 's* is the t -round interactive protocol $\Pi = (\mathcal{P}, \mathcal{V})$ where the prover $\mathcal{P}$ runs $\mathcal{P}_j$'s in parallel, i.e., if $\mathcal{P}_j$ sends $m_{i,j}$ during round i , then $\mathcal{P}$ sends $m_i := (m_{i,1}, \dots, m_{i,M})$ during round i . Similarly, $\mathcal{V}$ runs $\mathcal{V}_j$'s in parallel. Let p_j be the j -th “projection” function

Table 6: Distributed Batched FRI step-by-step cost analysis.

P_0 Runtime	P_i Runtime	Total comm. between P_0 and P_i	Protocol steps
$O(M)$	$O(T)$	$O(M)$	1. P_i builds the Merkle tree from its evaluation vector $(F_i(x))_{x \in D}$ and sends the commitment to P_0 . P_0 sends these commitments to V .
$O(M)$	$O(1)$	$O(M)$	2. P_0 receives a random challenge θ from V and sends it to P_i .
$O(N)$	$O(T)$	$O(N)$	3. P_i computes the new evaluation vector $(\theta^i F_i(x))_{x \in D}$ and sends it to P_0 .
$O(N)$	—	—	4. P_0 computes the evaluation vector $(F(x))_{x \in D}$ where $F(X) = \sum_i \theta^i F_i(X)$.
$O(T)$ for folding and $O(\log^2 T)$ for querying	—	—	5. P_0 and V run FRI with $F(X)$.
$O(M \log T)$	$O(\log T)$	$O(M \log T)$	6. Extra consistency checks for batched FRI: For each queried $g \in D$, P_i sends $F_i(g)$ and its Merkle authentication path to P_0 . P_0 sends all received data to V . V checks that the Merkle authentication paths are valid and $F(g) = \sum_{i=0}^{M-1} \theta^i F_i(g)$.
$O(N)$	$O(T)$	$O(N)$	Total

Table 7: Fold-and-Batch step-by-step cost analysis.

P_0	P_i	Total Communication	V	Protocol steps
$O(M)$	$O(T)$	$O(M)$	$O(M)$	Step 1
—	$O(T - K)$	—	—	Step 2
$O(M \log \frac{T}{K})$	$O(\log \frac{T}{K})$	$O(M \log \frac{T}{K})$	$O(M \log \frac{T}{K})$ from the $\log \frac{T}{K}$ Merkle commitments of intermediate folded polynomials from M parties.	Step 3
$O(M)$	$O(1)$	$O(M)$	—	Step 4
$O(MK)$	$O(K)$	$O(MK)$	—	Step 5
$O(MK)$	—	—	—	Step 6
$O(K)$	—	—	$O(\log K)$	Step 7
$O(M \log \frac{T}{K} \log T)$	$O(\log \frac{T}{K} \log T)$	$O(M \log \frac{T}{K} \log T)$	$O(M \log \frac{T}{K} \log T)$	Step 8
$O(\log^2 K)$	—	—	$O(\log^2 K)$	Step 9
—	—	—	$O(M)$ for (c)	Step 10
$O(MK + M \log \frac{T}{K} \log T)$	$O(T)$	$O(MK + M \log \frac{T}{K} \log T)$	$O(M \log \frac{T}{K} \log T + \log^2 K)$	Total

that takes as input a round- i transcript τ_i for Π and outputs a round- i transcript $\tau_{i,j}$ for Π_j as follows: if

$$\tau_i = (m_{1,1}, \dots, m_{1,M}) \parallel (c_{1,1}, \dots, c_{1,M}) \parallel \dots \parallel (m_{i,1}, \dots, m_{i,M}) \parallel (c_{i,1}, \dots, c_{i,M})$$

then

$$\tau_{i,j} := p_j(\tau_i) = m_{1,j} \parallel c_{1,j} \parallel \dots \parallel m_{i,j} \parallel c_{i,j}$$

If τ is a complete transcript for Π , then $\mathcal{V}$ accepts τ if and only if $\mathcal{V}_j$ accepts $p_j(\tau)$ for all $j \in [M]$.

Proposition 8. *The parallel execution of Π_j 's, Π , is a t -round IOP for the relation*

$$\mathcal{R} := \{(\mathbb{x} = (\mathbb{x}_1, \dots, \mathbb{x}_M), \mathbb{w} = (\mathbb{w}_1, \dots, \mathbb{w}_M)) \mid (\mathbb{x}_j, \mathbb{w}_j) \in \mathcal{R}_j \text{ for } j \in [M]\}$$

with δ -completeness and ε -soundness where $\delta = \prod_{j=1}^M \delta_j$ and $\varepsilon = \max_{1 \leq j \leq M} \varepsilon_j$.

Proof. δ -Completeness: Let $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$. Since $\mathcal{V}$ accepts if and only if all $\mathcal{V}_j$'s accept and the $\mathcal{V}_j$'s have independent random coins,

$$\begin{aligned} & \Pr[\langle \mathcal{P}(\mathbb{w}), \mathcal{V} \rangle(\mathbb{x}) = 1] \\ &= \Pr[\bigwedge_{j=1}^M \langle \mathcal{P}_j(\mathbb{w}_j), \mathcal{V}_j \rangle(\mathbb{x}_j) = 1] \\ &= \prod_{j=1}^M \Pr[\langle \mathcal{P}_j(\mathbb{w}_j), \mathcal{V}_j \rangle(\mathbb{x}_j) = 1] \\ &\geq \prod_{j=1}^M \delta_j = \delta \end{aligned}$$

ε -Soundness: Let $\mathbb{x} = (\mathbb{x}_1, \dots, \mathbb{x}_M) \notin \mathcal{L}_{\mathcal{R}}$, then some $\mathbb{x}_{j_0} \notin \mathcal{L}_{\mathcal{R}_{j_0}}$. Let $\mathcal{P}^*$ be any unbounded interactive algorithm and let $\mathcal{P}_j^*$ be the corresponding unbounded interactive algorithm for Π_j , i.e., the message sent by $\mathcal{P}_j^*$ during each round is the j -th entry of the message sent by $\mathcal{P}^*$ during that round. For the same reason as above, we have

$$\begin{aligned} & \Pr[\langle \mathcal{P}^*, \mathcal{V} \rangle(\mathbb{x}) = 1] \\ &= \Pr[\bigwedge_{j=1}^M \langle \mathcal{P}_j^*, \mathcal{V}_j \rangle(\mathbb{x}_j) = 1] \\ &= \prod_{j=1}^M \Pr[\langle \mathcal{P}_j^*, \mathcal{V}_j \rangle(\mathbb{x}_j) = 1] \\ &\leq \varepsilon_{j_0} \leq \max_{1 \leq j \leq M} \varepsilon_j = \varepsilon \end{aligned}$$

□